

Introducción al kit de herramientas MVVM

Artículo • 07/11/2024

El paquete `CommunityToolkit.Mvvm` (también conocido como kit de herramientas MVVM anteriormente denominado `Microsoft.Toolkit.Mvvm`) es una biblioteca MVVM moderna, rápida y modular. Forma parte del kit de herramientas de la comunidad de .NET y se basa en los siguientes principios:

- **Plataforma y Runtime independiente** - .NET Standard 2.0, .NET Standard 2.1 and .NET 6  (UI Framework independiente)
- **Recogida y uso sencillos** : no hay requisitos estrictos en la estructura de aplicaciones o los paradigmas de codificación (excepto "MVVM", es decir, uso flexible).
- **A la carta**: libertad para elegir qué componentes usar.
- **Implementación de referencia**: Lean y eficaz, proporciona implementaciones para interfaces incluidas en la biblioteca de clases base pero que carecen de tipos concretos para usarlas directamente.

Microsoft mantiene y publica el kit de herramientas MVVM y forma parte de .NET Foundation. También lo usan varias aplicaciones de primera entidad integradas en Windows, como [Microsoft Store](#).

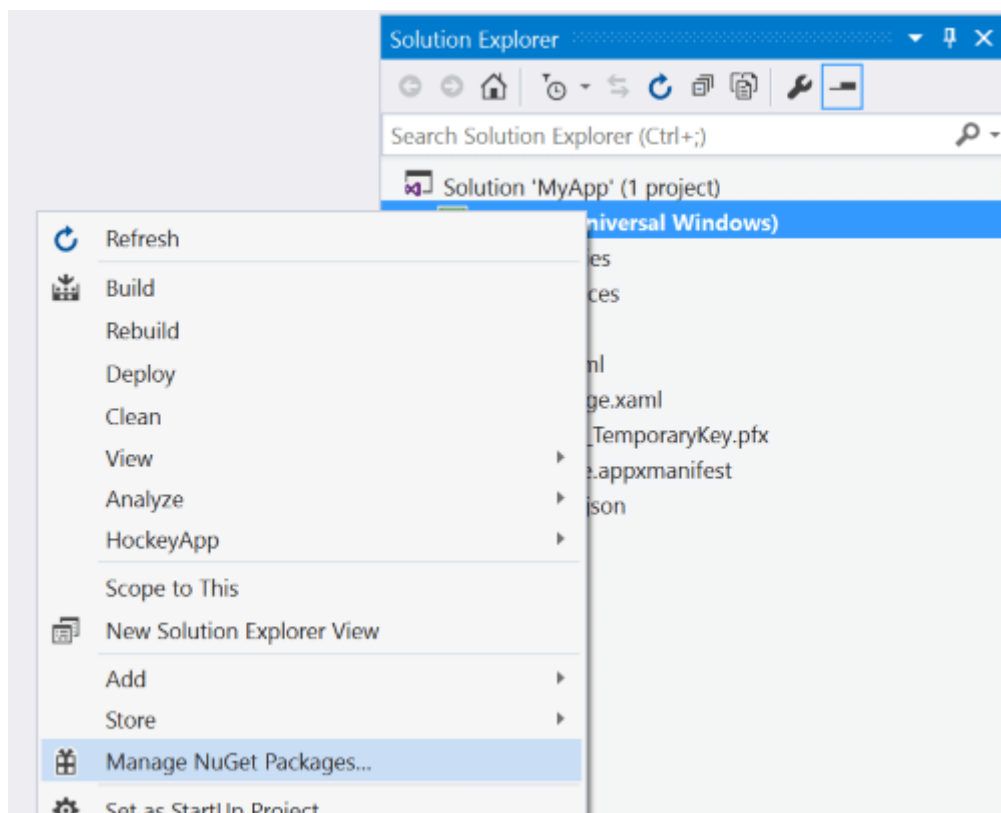
Este paquete tiene como destino **.NET Standard** para que se pueda usar en cualquier plataforma de aplicaciones: WinUI 3, UWP, WinForms, WPF, Xamarin, Uno, etc. y en cualquier entorno de ejecución: .NET Native, .NET Core, .NET Framework o Mono. Se ejecuta en todos ellos. La superficie de la API es idéntica en todos los casos, lo que hace que sea perfecta para crear bibliotecas compartidas.

Además, el kit de herramientas de MVVM también tiene un destino **.NET 6** que se usa para habilitar optimizaciones más internas al ejecutarse en .NET 6. La superficie de la API pública es idéntica en ambos casos, por lo que NuGet siempre resolverá la mejor versión posible del paquete sin que los consumidores tengan que preocuparse de qué API estarán disponibles en su plataforma.

Introducción

Para instalar el paquete desde Visual Studio:

1. En el Explorador de soluciones, haga clic con el botón derecho en el proyecto y seleccione **Administrar paquetes NuGet**. Busque **CommunityToolkit.Mvvm** e instálelo.



2. Agregue una directiva using o Imports para utilizar las nuevas API:

c#

```
using CommunityToolkit.Mvvm;
```

VB

```
Imports CommunityToolkit.Mvvm
```

3. Los ejemplos de código están disponibles en las otras páginas de documentos del kit de herramientas de MVVM y en las [pruebas unitarias](#) del proyecto.

¿Cuándo debería usar este paquete?

Use este paquete para acceder a una colección de tipos estándar, independientes y ligeros que proporcionan una implementación inicial para compilar aplicaciones modernas mediante el patrón MVVM. Estos tipos por sí solos suelen ser suficiente para que muchos usuarios compilen aplicaciones sin necesidad de referencias externas adicionales.

Los tipos incluidos son los siguientes:

- **CommunityToolkit.Mvvm.ComponentModel**
 - [ObservableObject](#)
 - [ObservableRecipient](#)

- [ObservableValidator](#)
- **CommunityToolkit.Mvvm.DependencyInjection**
 - [Ioc](#)
- **CommunityToolkit.Mvvm.Input**
 - [RelayCommand](#)
 - [RelayCommand<T>](#)
 - [AsyncRelayCommand](#)
 - [AsyncRelayCommand<T>](#)
 - [IRelayCommand](#)
 - [IRelayCommand<T>](#)
 - [IAsyncRelayCommand](#)
 - [IAsyncRelayCommand<T>](#)
- **CommunityToolkit.Mvvm.Messaging**
 - [IMessenger](#)
 - [WeakReferenceMessenger](#)
 - [StrongReferenceMessenger](#)
 - [IRecipient<TMessage>](#)
 - [MessageHandler<TRecipient, TMessage>](#)
- **CommunityToolkit.Mvvm.Messaging.Messages**
 - [PropertyChangedMessage<T>](#)
 - [RequestMessage<T>](#)
 - [AsyncRequestMessage<T>](#)
 - [CollectionRequestMessage<T>](#)
 - [AsyncCollectionRequestMessage<T>](#)
 - [ValueChangedMessage<T>](#)

Este paquete tiene como objetivo ofrecer toda la flexibilidad posible para que los desarrolladores puedan elegir qué componentes usar. Todos los tipos se acoplan de forma flexible, por lo que solo es necesario incluir lo que se usen. No hay ningún requisito para tener que comprometerse con una serie específica de API que abarque todo, ni existe un conjunto de patrones obligatorios que deban seguirse al compilar aplicaciones con estos asistentes. Combine estos bloques de creación de la manera que mejor se adapte a sus necesidades.

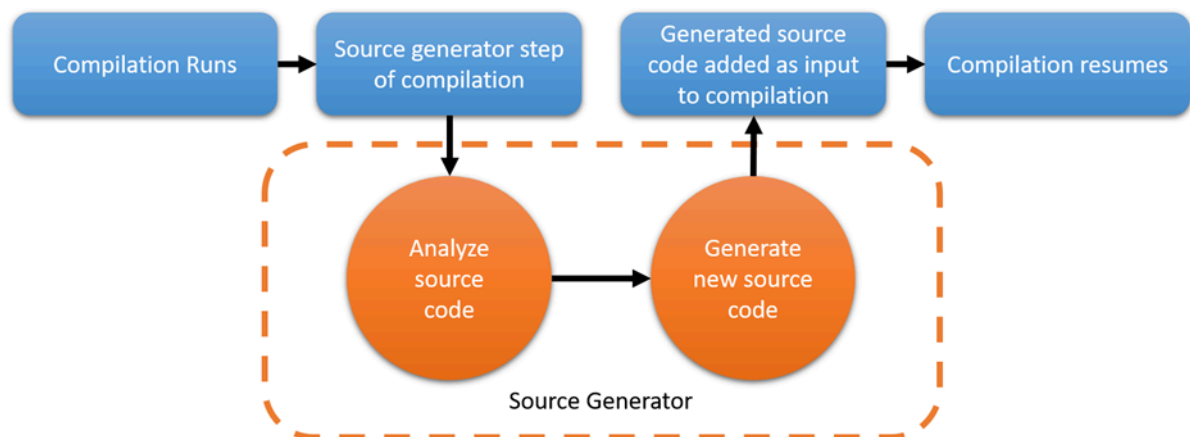
Recursos adicionales

- Consulte la [aplicación de ejemplo](#) [↗](#) (para varios marcos de interfaz de usuario) para ver el kit de herramientas de MVVM en acción.
- También puede encontrar más ejemplos en las [pruebas unitarias](#) [↗](#).

Generadores de origen de MVVM

Artículo • 07/11/2024

A partir de la versión 8.0, el kit de herramientas de MVVM incluye nuevos generadores de origen de Roslyn que ayudarán a reducir considerablemente la repetición de tareas al escribir código mediante la arquitectura de MVVM. Pueden simplificar escenarios en los que necesita configurar propiedades observables, comandos y mucho más. Si no está familiarizado con los generadores de origen, puede leer más sobre ellos [aquí](#). Esto es una vista simplificada de cómo funcionan:



Esto significa que, a medida que escribe código, el generador del kit de herramientas de MVVM ahora se encargará de generar código adicional para usted en segundo plano, por lo que no tiene que preocuparse por ello. Este código se compilará e incluirá en la aplicación, por lo que el resultado final es exactamente el mismo que si hubiera escrito manualmente todo ese código adicional, pero sin tener que hacer todo ese trabajo adicional. 🍌

Por ejemplo, sería genial si en lugar de tener que configurar una propiedad observable de forma normal:

C#

```
private string? name;

public string? Name
{
    get => name;
    set => SetProperty(ref name, value);
}
```

¿Podría tener un simple [campo anotado](#) para expresar lo mismo?

C#

```
[ObservableProperty]  
private string? name;
```

Qué pasa con la creación de un comando:

C#

```
private void SayHello()  
{  
    Console.WriteLine("Hello");  
}  
  
private ICommand? sayHelloCommand;  
  
public ICommand SayHelloCommand => sayHelloCommand ??= new  
RelayCommand(SayHello);
```

¿Qué pensaría si simplemente [pudiéramos tener nuestro método](#), y nada más?

C#

```
[RelayCommand]  
private void SayHello()  
{  
    Console.WriteLine("Hello");  
}
```

¡Con los nuevos generadores de origen de MVVM, todo esto es posible y mucho más!



ⓘ Nota

Los generadores de origen se pueden usar independientemente de otras características existentes en el kit de herramientas de MVVM, y puede mezclar y combinar generadores de origen con las API anteriores según sea necesario. Es decir, puede empezar a usar gradualmente generadores de código fuente en archivos nuevos y, con el tiempo, migrar archivos más antiguos para usarlos para reducir el nivel de detalle, pero no hay ninguna obligación de usar siempre un enfoque en todo un proyecto o aplicación.

Estos documentos repasarán exactamente qué características se incluyen con los generadores de MVVM y cómo usarlos:

- **CommunityToolkit.Mvvm.ComponentModel**
 - [ObservableProperty](#)
 - [INotifyPropertyChanged](#)
- **CommunityToolkit.Mvvm.Input**
 - [RelayCommand](#)

Ejemplos

- Consulte la [aplicación de ejemplo](#) [↗] (para varios marcos de interfaz de usuario) para ver el kit de herramientas de MVVM en acción.
- También puede encontrar más ejemplos en las [pruebas unitarias](#) [↗].

Atributo ObservableProperty

Artículo • 13/03/2024

El tipo [ObservableProperty](#) es un atributo que permite generar propiedades observables a partir de campos anotados. Su propósito es reducir considerablemente la cantidad de repeticiones que se necesitan para definir propiedades observables.

❗ Nota

Para poder funcionar, los campos anotados deben estar en una clase parcial con la infraestructura `INotifyPropertyChanged` necesaria. Si el tipo está anidado, todos los tipos del árbol de sintaxis de declaración también se deben anotar como parciales. Si no lo hace, se producirán errores de compilación, ya que el generador no podrá generar una declaración parcial diferente de ese tipo con la propiedad observable solicitada.

API de plataforma: [ObservableProperty](#), [NotifyPropertyChangedFor](#), [NotifyCanExecuteChangedFor](#), [NotifyDataErrorInfo](#), [NotifyPropertyChangedRecipients](#), [ICommand](#), [IRelayCommand](#), [ObservableValidator](#), [PropertyChangedMessage<T>](#), [IMessenger](#)

Funcionamiento

El atributo `ObservableProperty` se puede usar para anotar un campo en un tipo parcial, de la siguiente manera:

C#

```
[ObservableProperty]
private string? name;
```

Y generará una propiedad observable como esta:

C#

```
public string? Name
{
    get => name;
    set => SetProperty(ref name, value);
}
```

También lo hará con una implementación optimizada, por lo que el resultado final será aún más rápido.

❗ Nota

El nombre de la propiedad generada se creará en función del nombre del campo. El generador supone que el campo se denomina `lowerCamel`, `_lowerCamel` o `m_lowerCamel`, y transformará lo transformará a `UpperCamel` para seguir las convenciones de nomenclatura de .NET adecuadas. La propiedad resultante siempre tendrá descriptores de acceso públicos, pero el campo se puede declarar con cualquier visibilidad (se recomienda `private`).

Ejecución de código tras cambios

El código generado es realmente un poco más complejo que este, y el motivo de esto es que también expone algunos métodos que puede implementar para enlazar a la lógica de notificación y ejecutar lógica adicional cuando la propiedad está a punto de actualizarse y justo después de actualizarla, si es necesario. Es decir, el código generado es realmente similar al siguiente:

C#

```
public string? Name
{
    get => name;
    set
    {
        if (!EqualityComparer<string?>.Default.Equals(name, value))
        {
            string? oldValue = name;
            OnNameChanging(value);
            OnNameChanging(oldValue, value);
            OnPropertyChanging();
            name = value;
            OnNameChanged(value);
            OnNameChanged(oldValue, value);
            OnPropertyChanged();
        }
    }
}

partial void OnNameChanging(string? value);
partial void OnNameChanged(string? value);
```



```
partial void OnNameChanging(string? oldValue, string? newValue);
partial void OnNameChanged(string? oldValue, string? newValue);
```

Esto le permite implementar cualquiera de esos métodos para insertar código adicional. Los dos primeros son útiles siempre que quiera ejecutar alguna lógica que solo necesite hacer referencia al nuevo valor en el que se ha establecido la propiedad. Los otros dos son útiles siempre que tenga una lógica más compleja que también tenga que actualizar algún estado en el valor anterior y nuevo que se establece.

Por ejemplo, este es un ejemplo de cómo se pueden usar las dos primeras sobrecargas:

C#

```
[ObservableProperty]
private string? name;

partial void OnNameChanging(string? value)
{
    Console.WriteLine($"Name is about to change to {value}");
}

partial void OnNameChanged(string? value)
{
    Console.WriteLine($"Name has changed to {value}");
}
```

Y este es un ejemplo de cómo se pueden usar las otras dos sobrecargas:

C#

```
[ObservableProperty]
private ChildViewModel? selectedItem;

partial void OnSelectedItemChanging(ChildViewModel? oldValue,
ChildViewModel? newValue)
{
    if (oldValue is not null)
    {
        oldValue.IsSelected = true;
    }

    if (newValue is not null)
    {
        newValue.IsSelected = true;
    }
}
```

Solo tiene la libertad de implementar cualquier número de métodos entre los que están disponibles o ninguno de ellos. Si no se implementan (o si solo hay uno), el compilador

solo quitará todas las llamadas, por lo que no habrá ningún impacto de rendimiento en todos los casos en los que no se requiera esta funcionalidad adicional.

❗ Nota

Los métodos generados son métodos parciales sin implementación, lo que significa que si decide implementarlos, no puede especificar una accesibilidad explícita para ellos. Es decir, las implementaciones de estos métodos también deben declararse solo como métodos `partial`, y siempre tendrán implícitamente accesibilidad privada. Al intentar agregar una accesibilidad explícita (por ejemplo, agregar `public` o `private`) se producirá un error, ya que no se permite en C#.

Notificación de propiedades dependientes

Imagine que tenía una propiedad `FullName` para la que quería generar una notificación cada vez que `Name` cambia. Puede hacerlo mediante el atributo `NotifyPropertyChangedFor`, de la siguiente manera:

C#

```
[ObservableProperty]
[NotifyPropertyChangedFor(nameof(FullName))]
private string? name;
```

Esto dará como resultado una propiedad generada equivalente a esta:

C#

```
public string? Name
{
    get => name;
    set
    {
        if (SetProperty(ref name, value))
        {
            OnPropertyChanged("FullName");
        }
    }
}
```

Notificación de comandos dependientes

Imagine que tenía un comando cuyo estado de ejecución dependía del valor de esta propiedad. Es decir, siempre que la propiedad cambie, el estado de ejecución del comando se debe invalidar y calcular de nuevo. En otras palabras, [ICommand.CanExecuteChanged](#) se debe volver a generar. Puede lograrlo mediante el atributo `NotifyCanExecuteChangedFor`:

C#

```
[ObservableProperty]
[NotifyCanExecuteChangedFor(nameof(MyCommand))]
private string? name;
```

Esto dará como resultado una propiedad generada equivalente a esta:

C#

```
public string? Name
{
    get => name;
    set
    {
        if (SetProperty(ref name, value))
        {
            MyCommand.NotifyCanExecuteChanged();
        }
    }
}
```

Para que esto funcione, el comando de destino debe ser alguna propiedad [IRelayCommand](#).

Solicitud de validación de propiedades

Si la propiedad se declara en un tipo que hereda de [ObservableValidator](#), también es posible anotarla con cualquier atributo de validación y, a continuación, solicitar al establecedor generado que desencadene la validación de esa propiedad. Esto se puede lograr con el atributo `NotifyDataErrorInfo`:

C#

```
[ObservableProperty]
[NotifyDataErrorInfo]
[Required]
[MinLength(2)] // Any other validation attributes too...
private string? name;
```

Esto hará que se genere la siguiente propiedad:

```
C#

public string? Name
{
    get => name;
    set
    {
        if (SetProperty(ref name, value))
        {
            ValidateProperty(value, "Value2");
        }
    }
}
```

Esa llamada generada `ValidateProperty` validará la propiedad y actualizará el estado del objeto `ObservableValidator` para que los componentes de la interfaz de usuario puedan reaccionar a ella y mostrar los errores de validación correctamente.

⚠ Nota

Por diseño, solo los atributos de campo que heredan de [ValidationAttribute](#) se reenviarán a la propiedad generada. Esto se hace específicamente para admitir escenarios de validación de datos. Todos los demás atributos de campo se omitirán, por lo que actualmente no es posible agregar atributos personalizados adicionales en un campo y tenerlos también aplicados a la propiedad generada. Si es necesario (por ejemplo, para controlar la serialización), considere la posibilidad de usar una propiedad manual tradicional en su lugar.

Envío de mensajes de notificación

Si la propiedad se declara en un tipo que hereda de [ObservableRecipient](#), puede usar el atributo `NotifyPropertyChangedRecipients` para indicar al generador que también inserte código para enviar un mensaje cambiado de propiedad para el cambio de propiedad. Esto permitirá que los destinatarios registrados reaccionen dinámicamente al cambio. Es decir, tenga en cuenta este código:

```
C#

[ObservableProperty]
[NotifyPropertyChangedRecipients]
private string? name;
```

Esto hará que se genere la siguiente propiedad:

```
C#  
  
public string? Name  
{  
    get => name;  
    set  
    {  
        string? oldValue = name;  
  
        if (SetProperty(ref name, value))  
        {  
            Broadcast(oldValue, value);  
        }  
    }  
}
```

Después, esa llamada generada `Broadcast` enviará un nuevo `PropertyChangedMessage<T>` que usa la instancia `IMessenger` en uso en el modelo de vista actual a todos los suscriptores registrados.

Adición de atributos personalizados

En algunos casos, puede resultar útil tener también algunos atributos personalizados sobre las propiedades generadas. Para lograrlo, simplemente puede usar el destino `[property: ]` en las listas de atributos sobre campos anotados y el kit de herramientas de MVVM reenvía automáticamente esos atributos a las propiedades generadas.

Por ejemplo, considere un campo similar al siguiente:

```
C#  
  
[ObservableProperty]  
[property: JsonRequired]  
[property: JsonPropertyName("name")]  
private string? username;
```

Esto generará una propiedad `Username`, con esos dos atributos `[JsonRequired]` y `[JsonPropertyName("name")]` sobre ella. Puede usar tantas listas de atributos que tienen como destino la propiedad como desee y todas ellas se reenviarán a las propiedades generadas.

Ejemplos

- Consulte la [aplicación de ejemplo](#) (para varios marcos de interfaz de usuario) para ver el kit de herramientas de MVVM en acción.
- También puede encontrar más ejemplos en las [pruebas unitarias](#).

Colaborar con nosotros en GitHub

El origen de este contenido se puede encontrar en GitHub, donde también puede crear y revisar problemas y solicitudes de incorporación de cambios. Para más información, consulte [nuestra guía para colaboradores](#).

.NET

Comentarios de MVVM Toolkit

MVVM Toolkit es un proyecto de código abierto. Seleccione un vínculo para proporcionar comentarios:

 [Abrir incidencia con la documentación](#)

 [Proporcionar comentarios sobre el producto](#)

Atributo RelayCommand

Artículo • 17/03/2024

El tipo [RelayCommand](#) es un atributo que permite generar propiedades de comando de retransmisión para métodos anotados. Su propósito es eliminar completamente la reutilizable que se necesita para definir comandos encapsulando métodos privados en un modelo de vista.

❗ Nota

Para poder funcionar, los métodos anotados deben estar en una clase parcial. Si el tipo está anidado, todos los tipos del árbol de sintaxis de declaración también se deben anotar como parciales. Si no lo hace, se producirán errores de compilación, ya que el generador no podrá generar una declaración parcial diferente de ese tipo con el comando solicitado.

API de la plataforma: [RelayCommand](#), [ICommand](#), [IRelayCommand](#), [IRelayCommand<T>](#), [IAsyncRelayCommand](#), [IAsyncRelayCommand<T>](#), [Task](#), [CancellationToken](#)

Funcionamiento

El atributo `RelayCommand` se puede usar para anotar un método en un tipo parcial, de la siguiente manera:

C#

```
[RelayCommand]
private void GreetUser()
{
    Console.WriteLine("Hello!");
}
```

Y generará un comando similar al siguiente:

C#

```
private RelayCommand? greetUserCommand;

public IRelayCommand GreetUserCommand => greetUserCommand ??= new
RelayCommand(GreetUser);
```

⚠ Nota

El nombre del comando generado se creará en función del nombre del método. El generador usará el nombre del método y anexará "Command" al final, y quitará el prefijo "On", si está presente. Adicionalmente, para los métodos asíncronos, el sufijo "Async" también se elimina antes de añadir "Command".

Parámetros de comando

El atributo `[RelayCommand]` admite la creación de comandos para métodos con un parámetro. En ese caso, cambiará automáticamente el comando generado para que sea en su lugar `IRelayCommand<T>`, aceptando un parámetro del mismo tipo:

```
C#  
  
[RelayCommand]  
private void GreetUser(User user)  
{  
    Console.WriteLine($"Hello {user.Name}!");  
}
```

Esto dará como resultado el código generado siguiente:

```
C#  
  
private RelayCommand<User>? greetUserCommand;  
  
public IRelayCommand<User> GreetUserCommand => greetUserCommand ??= new  
    RelayCommand<User>(GreetUser);
```

El comando resultante usará automáticamente el tipo del argumento como argumento de tipo.

Comandos asíncronos

El comando `[RelayCommand]` también admite el ajuste de métodos asíncronos a través de las interfaces `IAsyncRelayCommand` y `IAsyncRelayCommand<T>`. Esto se controla automáticamente cada vez que un método devuelve un tipo `Task`. Por ejemplo:

```
C#
```



```
[RelayCommand]
private async Task GreetUserAsync()
{
    User user = await userService.GetCurrentUserAsync();

    Console.WriteLine($"Hello {user.Name}!");
}
```

Esto dará como resultado el código siguiente:

C#

```
private AsyncRelayCommand? greetUserCommand;

public IAsyncRelayCommand GreetUserCommand => greetUserCommand ??= new
AsyncRelayCommand(GreetUserAsync);
```

Si el método toma un parámetro, el comando resultante también será genérico.

Hay un caso especial cuando el método tiene un [CancellationToken](#), ya que se propagará al comando para habilitar la cancelación. Es decir, un método similar al siguiente:

C#

```
[RelayCommand]
private async Task GreetUserAsync(Cancellation token)
{
    try
    {
        User user = await userService.GetCurrentUserAsync(token);

        Console.WriteLine($"Hello {user.Name}!");
    }
    catch (OperationCanceledException)
    {
    }
}
```

Dará como resultado que el comando generado pase un token al método ajustado. Esto permite a los consumidores llamar a [IAsyncRelayCommand.Cancel](#) para indicar ese token y permitir que las operaciones pendientes se detengan correctamente.

Habilitación y deshabilitación de comandos

A menudo resulta útil poder deshabilitar comandos y después, invalidar su estado y volver a comprobar si se pueden ejecutar o no. Para admitir esto, el atributo `RelayCommand` expone la propiedad `CanExecute`, que se puede usar para indicar una propiedad o método de destino que se va a usar para evaluar si se puede ejecutar un comando:

C#

```
[RelayCommand(CanExecute = nameof(CanGreetUser))]  
private void GreetUser(User? user)  
{  
    Console.WriteLine($"Hello {user!.Name}!");  
}  
  
private bool CanGreetUser(User? user)  
{  
    return user is not null;  
}
```

De este modo, se invoca `CanGreetUser` cuando el botón se enlaza primero a la interfaz de usuario (por ejemplo, a un botón) y a continuación, se invoca `IRelayCommand.NotifyCanExecuteChanged` de nuevo cada vez que se invoca en el comando.

Por ejemplo, este es el modo en que un comando se puede enlazar a una propiedad para controlar su estado:

C#

```
[ObservableProperty]  
[NotifyCanExecuteChangedFor(nameof(GreetUserCommand))]  
private User? selectedUser;
```

XML

```
<!-- Note: this example uses traditional XAML binding syntax -->  
<Button  
    Content="Greet user"  
    Command="{Binding GreetUserCommand}"  
    CommandParameter="{Binding SelectedUser}"/>
```

En este ejemplo, la propiedad `SelectedUser` generada invocará el método `GreetUserCommand.NotifyCanExecuteChanged()` cada vez que cambie su valor. La interfaz de usuario tiene un enlace de control `Button` a `GreetUserCommand`, lo que significa que cada vez que se genera su evento `CanExecuteChanged`, llamará a su método `CanExecute`

de nuevo. Esto hará que se evalúe el método `CanGreetUser` encapsulado, que devolverá el nuevo estado del botón en función de si la instancia `User` de entrada (que en la interfaz de usuario está vinculada a la propiedad `SelectedUser`) lo está `null` o no. Esto significa que, cada vez que `SelectedUser` se cambia, `GreetUserCommand` se habilitará o no en función de si esa propiedad tiene un valor, que es el comportamiento deseado en este escenario.

ⓘ Nota

El comando **no** conocerá automáticamente cuándo ha cambiado el valor devuelto para el método `CanExecute` o la propiedad. Es necesario que el desarrollador llame `IRelayCommand.NotifyCanExecuteChanged` para invalidar el comando y solicitar que el método vinculado `CanExecute` se evalúe de nuevo para actualizar el estado visual del control enlazado al comando.

Control de ejecuciones simultáneas

Cada vez que un comando es asíncrono, se puede configurar para decidir si se permiten ejecuciones simultáneas o no. Al usar el atributo `RelayCommand`, se puede establecer a través de la propiedad `AllowConcurrentExecutions`. El valor predeterminado es `false`, lo que significa que hasta que una ejecución está pendiente, el comando indicará su estado como deshabilitado. Si en su lugar se establece en `true`, se puede poner en cola cualquier número de invocaciones simultáneas.

Tenga en cuenta que si un comando acepta un token de cancelación, también se cancelará un token si se solicita una ejecución simultánea. La principal diferencia es que si se permiten ejecuciones simultáneas, el comando permanecerá habilitado y iniciará una nueva ejecución solicitada sin esperar a que se complete realmente la anterior.

Control de excepciones asíncronas

Hay dos maneras diferentes de controlar las excepciones de los comandos de retransmisión asíncrona:

- **Esperar y volver a lanzar (valor predeterminado):** cuando el comando espera la finalización de una invocación, cualquier excepción se lanzará naturalmente en el mismo contexto de sincronización. Esto suele significar que las excepciones que se producen simplemente bloquearían la aplicación, que es un comportamiento

coherente con el de los comandos sincrónicos (donde las excepciones que se producen también bloquearán la aplicación).

- **Excepciones de flujo al programador de tareas:** si un comando está configurado para fluir excepciones al programador de tareas, las excepciones que se producen no bloquearán la aplicación, sino que ambas estarán disponibles a través de la expuesta [IAsyncRelayCommand.ExecutionTask](#) así como la propagación hasta [TaskScheduler.UnobservedTaskException](#). Esto permite escenarios más avanzados (por ejemplo, tener componentes de interfaz de usuario enlazados a la tarea y mostrar resultados diferentes en función del resultado de la operación), pero es más complejo usar correctamente.

El comportamiento predeterminado es tener comandos await y volver a iniciar excepciones. Esto se puede configurar a través de la propiedad

`FlowExceptionsToTaskScheduler`:

C#

```
[RelayCommand(FlowExceptionsToTaskScheduler = true)]
private async Task GreetUserAsync(Cancellation token)
{
    User user = await userService.GetCurrentUserAsync(token);

    Console.WriteLine($"Hello {user.Name}!");
}
```

En este caso, `try/catch` ya no es necesario, ya que las excepciones ya no bloquearán la aplicación. Tenga en cuenta que esto también hará que otras excepciones no relacionadas no se vuelvan a iniciar automáticamente, por lo que debe decidir cuidadosamente cómo abordar cada escenario individual y configurar el resto del código correctamente.

Cancelar comandos para operaciones asincrónicas

Una última opción para los comandos asincrónicos es la capacidad de solicitar que se genere un comando cancel. Se trata de un ajuste `ICommand` un comando de retransmisión asincrónica que se puede usar para solicitar la cancelación de una operación. Este comando indicará automáticamente su estado para reflejar si se puede usar o no en un momento dado. Por ejemplo, si el comando vinculado no se está ejecutando, notificará su estado como tampoco ejecutable. Esto se puede usar de la siguiente manera:

C#

```
[RelayCommand(IncludeCancelCommand = true)]
private async Task DoWorkAsync(Cancellation token)
{
    // Do some long running work...
}
```

Esto hará que también se genere una propiedad `DoWorkCancelCommand`. Después, esto se puede enlazar a algún otro componente de interfaz de usuario para permitir que los usuarios cancelen fácilmente las operaciones asíncronas pendientes.

Adición de atributos personalizados

Al igual que con las [propiedades observables](#), el `RelayCommand` generador también incluye compatibilidad con atributos personalizados para las propiedades generadas. Para aprovechar esto, simplemente puede usar el destino `[property: ]` en las listas de atributos sobre métodos anotados y el kit de herramientas de MVVM reenvía esos atributos a las propiedades de comando generadas.

Por ejemplo, considere un método similar al siguiente:

C#

```
[RelayCommand]
[property: JsonIgnore]
private void GreetUser(User user)
{
    Console.WriteLine($"Hello {user.Name}!");
}
```

Esto generará una propiedad `GreetUserCommand`, con el atributo `[JsonIgnore]` sobre él. Puede usar tantas listas de atributos que tienen como destino la propiedad como desee y todas ellas se reenviarán a las propiedades generadas.

Ejemplos

- Consulte la [aplicación de ejemplo](#) (para varios marcos de interfaz de usuario) para ver el kit de herramientas de MVVM en acción.
- También puede encontrar más ejemplos en las [pruebas unitarias](#).

Colaborar con nosotros en GitHub

El origen de este contenido se puede encontrar en GitHub, donde también puede crear y revisar problemas y solicitudes de incorporación de cambios. Para más información, consulte [nuestra guía para colaboradores](#).

.NET

Comentarios de MVVM Toolkit

MVVM Toolkit es un proyecto de código abierto. Seleccione un vínculo para proporcionar comentarios:

 [Abrir incidencia con la documentación](#)

 [Proporcionar comentarios sobre el producto](#)

Atributos INotifyPropertyChanged

Artículo • 07/11/2024

El tipo [INotifyPropertyChanged](#) es un atributo que permite insertar código compatible con MVVM en tipos existentes. Junto con otros atributos relacionados ([ObservableObject](#) y [ObservableRecipient](#)), su propósito es ayudar a los desarrolladores en los casos en los que se necesitaba la misma funcionalidad de estos tipos, pero los tipos de destino ya la estaban implementando a partir de otro tipo. Dado que C# no permite varias herencias, estos atributos se pueden usar en su lugar para que el generador del kit de herramientas de MVVM agregue el mismo código directamente a esos tipos, recortando esta limitación.

ⓘ Nota

Para poder funcionar, los tipos anotados deben estar en una [clase parcial](#). Si el tipo está anidado, todos los tipos del árbol de sintaxis de declaración también se deben anotar como parciales. Si no lo hace, se producirán errores de compilación, ya que el generador no podrá generar una declaración parcial diferente de ese tipo con el código adicional solicitado.

ⓘ Nota

Estos atributos solo están diseñados para usarse en los casos en los que los tipos de destino no pueden heredar simplemente de los tipos equivalentes (por ejemplo, de `ObservableObject`). Si es posible, heredar es el enfoque recomendado, ya que reducirá el tamaño binario evitando la creación de código duplicado en el ensamblado final.

API de la plataforma: [INotifyPropertyChanged](#), [ObservableObject](#), [ObservableRecipient](#)

Cómo usarlos

El uso de cualquiera de estos atributos es bastante sencillo: simplemente agréuelos a una [clase parcial](#) y todo el código de los tipos correspondientes se generará automáticamente en ese tipo. Por ejemplo, considere esta canalización:

C#

```
[INotifyPropertyChanged]
public partial class MyViewModel : SomeOtherType
{
}
```

Esto generará una implementación de `INotifyPropertyChanged` en el tipo `MyViewModel`, completa con asistentes adicionales (como `SetProperty`) que se pueden usar para reducir el nivel de detalle. Este es un breve resumen de los distintos atributos:

- **`INotifyPropertyChanged`**: implementa la interfaz y agrega métodos auxiliares para establecer propiedades y generar los eventos.
- **`ObservableObject`**: agrega todo el código del tipo `ObservableObject`. Es conceptualmente equivalente a `INotifyPropertyChanged`, con la principal diferencia de que también implementa `INotifyPropertyChanging`.
- **`ObservableRecipient`**: agrega todo el código del tipo `ObservableRecipient`. En concreto, esto se puede agregar a un tipo que hereda de `ObservableValidator` para combinar los dos.

Ejemplos

- Consulte la [aplicación de ejemplo](#) (para varios marcos de interfaz de usuario) para ver el kit de herramientas de MVVM en acción.
- También puede encontrar más ejemplos en las [pruebas unitarias](#).

ObservableObject

Artículo • 07/11/2024

El [ObservableObject](#) es una clase base para los objetos que son observables mediante la implementación de las interfaces [INotifyPropertyChanged](#) y [INotifyPropertyChanging](#). Se puede usar como punto de partida para todos los tipos de objetos que necesitan admitir notificaciones de cambio de propiedad.

API de la plataforma: [ObservableObject](#), [TaskNotifier](#), [TaskNotifier<T>](#)

Funcionamiento

`ObservableObject` tiene las siguientes características principales:

- Proporciona una implementación base para `INotifyPropertyChanged` y `INotifyPropertyChanging`, que expone los eventos `PropertyChanged` y `PropertyChanging`.
- Proporciona una serie de métodos de `SetProperty` que se pueden usar para establecer fácilmente valores de propiedad de los tipos que heredan de `ObservableObject`, y para generar automáticamente los eventos adecuados.
- Proporciona el método `SetPropertyAndNotifyOnCompletion`, que es análogo a `SetProperty` pero con la capacidad de establecer propiedades `Task` y generar automáticamente los eventos de notificación cuando se completan las tareas asignadas.
- Expone los métodos `OnPropertyChanged` y `OnPropertyChanging`, que se pueden invalidar en tipos derivados para personalizar cómo se generan los eventos de notificación.

Propiedad simple

Este es un ejemplo de cómo implementar la compatibilidad con notificaciones en una propiedad personalizada:

C#

```
public class User : ObservableObject
{
    private string name;

    public string Name
    {
        get => name;
        set => SetProperty(ref name, value);
    }
}
```

El método `SetProperty<T>(ref T, T, string)` proporcionado comprueba el valor actual de la propiedad y lo actualiza si es diferente y, a continuación, también genera automáticamente los eventos pertinentes. El nombre de la propiedad se captura automáticamente mediante el uso del atributo `[CallerMemberName]`, por lo que no es necesario especificar manualmente qué propiedad se está actualizando.

Ajuste de un modelo no observable

Un escenario común, por ejemplo, cuando se trabaja con elementos de base de datos, consiste en crear un modelo "enlazable" de ajuste que retransmite las propiedades del modelo de base de datos y genera las notificaciones modificadas de la propiedad cuando es necesario. Esto también es necesario cuando se desea insertar compatibilidad con notificaciones en los modelos, que no implementan la interfaz `INotifyPropertyChanged`. `ObservableObject` proporciona un método dedicado para simplificar este proceso. En el ejemplo siguiente, `User` es un modelo que asigna directamente una tabla de base de datos, sin heredar de `ObservableObject`:

C#

```
public class ObservableUser : ObservableObject
{
    private readonly User user;

    public ObservableUser(User user) => this.user = user;

    public string Name
    {
        get => user.Name;
        set => SetProperty(user.Name, value, user, (u, n) => u.Name = n);
    }
}
```

En este caso, se usa la sobrecarga `SetProperty<TModel, T>(T, T, TModel, Action<TModel, T>, string)`. La firma es ligeramente más compleja que la anterior: esto es necesario para permitir que el código siga siendo extremadamente eficaz aunque no tengamos acceso a un campo de respaldo como en el escenario anterior. Podemos pasar por cada parte de esta firma de método en detalle para comprender el rol de los distintos componentes:

- `TModel` es un argumento de tipo, que indica el tipo del modelo que estamos ajustando. En este caso, será nuestra clase `User`. Tenga en cuenta que no es necesario especificar esto explícitamente: el compilador de C# lo deducirá automáticamente mediante la invocación del método `SetProperty`.

- `T` es el tipo de la propiedad que queremos establecer. De forma similar a `TModel`, esto se deduce automáticamente.
- `T oldValue` es el primer parámetro y, en este caso, usamos `user.Name` para pasar el valor actual de esa propiedad que estamos encapsulando.
- `T newValue` es el nuevo valor que se va a establecer en la propiedad y aquí se pasa `value`, que es el valor de entrada dentro del establecedor de propiedades.
- `TModel model` es el modelo de destino que estamos encapsulando, en este caso se pasa la instancia almacenada en el campo `user`.
- `Action<TModel, T> callback` es una función que se invocará si el nuevo valor de la propiedad es diferente al actual y la propiedad debe establecerse. Esto lo hará esta función de devolución de llamada, que recibe como entrada el modelo de destino y el nuevo valor de propiedad que se va a establecer. En este caso, simplemente asignamos el valor de entrada (que llamamos `n`) a la `Name` propiedad (haciendo `u.Name = n`). Aquí es importante evitar capturar valores del ámbito actual e interactuar solo con los proporcionados como entrada a la devolución de llamada, ya que esto permite al compilador de C# almacenar en caché la función de devolución de llamada y realizar una serie de mejoras de rendimiento. Esto se debe a que no solo estamos accediendo directamente al campo `user` aquí o al parámetro `value` en el establecedor, sino que solo usamos los parámetros de entrada para la expresión lambda.

El método `SetProperty<TModel, T>(T, T, TModel, Action<TModel, T>, string)` hace que la creación de estas propiedades de ajuste sea extremadamente sencilla, ya que se encarga de recuperar y establecer las propiedades de destino al tiempo que proporciona una API extremadamente compacta.

⚠ Nota

En comparación con la implementación de este método mediante expresiones LINQ, específicamente a través de un parámetro de tipo `Expression<Func<T>>` en lugar de los parámetros de estado y devolución de llamada, las mejoras de rendimiento que se pueden lograr de esta manera son realmente significativas. En concreto, esta versión es ~200x más rápida que la que usa expresiones LINQ y no realiza asignaciones de memoria en absoluto.

Control de propiedades `Task<T>`

Si una propiedad es un `Task` también es necesario generar el evento de notificación una vez completada la tarea, de modo que los enlaces se actualicen en el momento adecuado.

Ejemplo, para mostrar un indicador de carga u otra información de estado en la operación representada por la tarea. `ObservableObject` tiene una API para este escenario:

C#

```
public class MyModel : ObservableObject
{
    private TaskNotifier<int>? requestTask;

    public Task<int>? RequestTask
    {
        get => requestTask;
        set => SetPropertyAndNotifyOnCompletion(ref requestTask, value);
    }

    public void RequestValue()
    {
        RequestTask = WebService.LoadMyValueAsync();
    }
}
```

Aquí el método `SetPropertyAndNotifyOnCompletion<T>(ref TaskNotifier<T>, Task<T>, string)` se encargará de actualizar el campo de destino, supervisar la nueva tarea, si está presente y generar el evento de notificación cuando se complete esa tarea. De este modo, es posible enlazar solo a una propiedad de tarea y recibir una notificación cuando cambie su estado. El `TaskNotifier<T>` es un tipo especial expuesto por `ObservableObject` que encapsula una instancia `Task<T>` de destino y habilita la lógica de notificación necesaria para este método. El tipo `TaskNotifier` también está disponible para usarlo directamente si solo tiene una `Task` general.

❗ Nota

El método `SetPropertyAndNotifyOnCompletion` está diseñado para reemplazar el uso del tipo `NotifyTaskCompletion<T>` del paquete de `Microsoft.Toolkit`. Si se usaba este tipo, se puede reemplazar solo por la propiedad `Task` interna (o `Task<TResult>`) y, a continuación, se puede usar el método `SetPropertyAndNotifyOnCompletion` para establecer su valor y generar cambios de notificación. Todas las propiedades expuestas por el tipo de `NotifyTaskCompletion<T>` están disponibles directamente en `Task` instancias.

Ejemplos

- Consulte la [aplicación de ejemplo](#) [↗](#) (para varios marcos de interfaz de usuario) para ver el kit de herramientas de MVVM en acción.

- También puede encontrar más ejemplos en las [pruebas unitarias](#).

ObservableRecipient

Artículo • 13/03/2024

El tipo `ObservableRecipient` es una clase base para objetos observables que también actúa como destinatarios para los mensajes. Esta clase es una extensión de `ObservableObject`, que también proporciona compatibilidad integrada para usar el tipo `IMessenger`.

API de la plataforma: `ObservableRecipient`, `ObservableObject`, `IMessenger`, `WeakReferenceMessenger`, `IRecipient<TMessage>`, `PropertyChangedMessage<T>`

Funcionamiento

El tipo `ObservableRecipient` está diseñado para usarse como base para los modelos de vista que también usan las características `IMessenger`, ya que proporciona compatibilidad integrada para él. En concreto:

- Tiene un constructor sin parámetros y otro que toma una instancia `IMessenger`, que se usará con la inserción de dependencias. También expone una propiedad `Messenger` que se puede usar para enviar y recibir mensajes en el modelo de vista. Si se usa el constructor sin parámetros, la instancia `WeakReferenceMessenger.Default` se asignará a la propiedad `Messenger`.
- Expone una propiedad `IsActive` para activar o desactivar el modelo de vista. En este contexto, "activar" significa que un modelo de vista determinado está marcado como en uso, por ejemplo, comenzará a escuchar mensajes registrados, realizar otras operaciones de configuración, etc. Hay dos métodos relacionados, `OnActivated` y `OnDeactivated`, que se invocan cuando cambia el valor de la propiedad. De forma predeterminada, `OnDeactivated` anula automáticamente el registro de la instancia actual de todos los mensajes registrados. Para obtener los mejores resultados y evitar pérdidas de memoria, se recomienda usar `OnActivated` para registrarse en los mensajes y usar `OnDeactivated` para realizar operaciones de limpieza. Este patrón permite que un modelo de vista se habilite o deshabilite varias veces, mientras que es seguro recopilarlo sin el riesgo de pérdidas de memoria cada vez que se desactiva. De forma predeterminada, `OnActivated` registrará automáticamente todos los controladores de mensajes definidos a través de la interfaz `IRecipient<TMessage>`.
- Expone un método `Broadcast<T>(T, T, string)` que envía un mensaje `PropertyChangedMessage<T>` a través de la instancia `IMessenger` disponible desde la propiedad `Messenger`. Esto se puede usar para difundir fácilmente los cambios en las propiedades de un modelo de vista sin tener que recuperar manualmente una instancia `Messenger` que se va a usar. La sobrecarga de los distintos métodos `SetProperty` usa este

método, que tiene una propiedad adicional `bool broadcast` para indicar si también se va a enviar un mensaje.

Este es un ejemplo de un modelo de vista que recibe mensajes `LoggedInUserRequestMessage` cuando está activo:

C#

```
public class MyViewModel : ObservableRecipient,
    IRecipient<LoggedInUserRequestMessage>
{
    public void Receive(LoggedInUserRequestMessage message)
    {
        // Handle the message here
    }
}
```

En el ejemplo anterior, `OnActivated` registra automáticamente la instancia como un destinatario para los mensajes `LoggedInUserRequestMessage`, utilizando ese método como la acción que se va a invocar. El uso de la interfaz `IRecipient<TMessage>` no es obligatorio y el registro también se puede realizar manualmente (incluso usando solo una expresión lambda insertada):

C#

```
public class MyViewModel : ObservableRecipient
{
    protected override void OnActivated()
    {
        // Using a method group...
        Messenger.Register<MyViewModel, LoggedInUserRequestMessage>(this, (r, m)
=> r.Receive(m));

        // ...or a lambda expression
        Messenger.Register<MyViewModel, LoggedInUserRequestMessage>(this, (r, m)
=>
        {
            // Handle the message here
        }));
    }

    private void Receive(LoggedInUserRequestMessage message)
    {
        // Handle the message here
    }
}
```

Ejemplos

- Consulte la [aplicación de ejemplo](#) (para varios marcos de interfaz de usuario) para ver el kit de herramientas de MVVM en acción.
- También puede encontrar más ejemplos en las [pruebas unitarias](#).

ObservableValidator

Artículo • 07/11/2024

El [ObservableValidator](#) es una clase base que implementa la interfaz [INotifyDataErrorInfo](#), lo que proporciona compatibilidad para validar las propiedades expuestas a otros módulos de aplicación. También hereda de `ObservableObject`, por lo que también implementa [INotifyPropertyChanged](#) y [INotifyPropertyChanging](#). Se puede usar como punto de partida para todos los tipos de objetos que necesitan admitir las notificaciones de cambio de propiedad y la validación de propiedades.

API de plataforma: [ObservableValidator](#), [ObservableObject](#)

Funcionamiento

`ObservableValidator` tiene las siguientes características principales:

- Proporciona una implementación base para `INotifyDataErrorInfo`, que expone el evento `ErrorsChanged` y las otras API necesarias.
- Proporciona una serie de sobrecargas `SetProperty` adicionales (sobre las proporcionadas por la clase base `ObservableObject`), que ofrecen la capacidad de validar automáticamente las propiedades y generar los eventos necesarios antes de actualizar sus valores.
- Expone una serie de sobrecargas `TrySetProperty`, que son similares a `SetProperty` pero con la capacidad de actualizar solo la propiedad de destino si la validación se realiza correctamente y devolver los errores generados (si los hay) para una inspección adicional.
- Expone el método `ValidateProperty`, que puede ser útil para desencadenar manualmente la validación de una propiedad específica en caso de que su valor no se haya actualizado, pero su validación depende del valor de otra propiedad que se haya actualizado en su lugar.
- Expone el método `ValidateAllProperties`, que ejecuta automáticamente la validación de todas las propiedades de instancia pública de la instancia actual, siempre que tengan al menos un [\[ValidationAttribute\]](#) aplicado a ellas.
- Expone un método `ClearAllErrors` que puede ser útil al restablecer un modelo enlazado a algún formulario que el usuario podría querer rellenar de nuevo.
- Ofrece una serie de constructores que permiten pasar parámetros diferentes para inicializar la instancia [ValidationContext](#) que se usará para validar las propiedades. Esto puede ser especialmente útil cuando se usan atributos de validación personalizados que podrían requerir servicios o opciones adicionales para funcionar correctamente.

Propiedad simple

Este es un ejemplo de cómo implementar una propiedad que admita las notificaciones de cambio, así como la validación:

C#

```
public class RegistrationForm : ObservableValidator
{
    private string name;

    [Required]
    [MinLength(2)]
    [MaxLength(100)]
    public string Name
    {
        get => name;
        set => SetProperty(ref name, value, true);
    }
}
```

Aquí llamamos al método `SetProperty<T>(ref T, T, bool, string)` expuesto por `ObservableValidator`, y ese parámetro `bool` adicional establecido en `true` indica que también queremos validar la propiedad cuando se actualiza su valor. `ObservableValidator` ejecutará automáticamente la validación en cada nuevo valor mediante todas las comprobaciones especificadas con los atributos aplicados a la propiedad. Otros componentes (como los controles de interfaz de usuario) pueden interactuar con el modelo de vista y modificar su estado para reflejar los errores presentes actualmente en el modelo de vista, registrando en `ErrorsChanged` y usando el método `GetErrors(string)` para recuperar la lista de errores de cada propiedad que se ha modificado.

Métodos de validación personalizados

A veces, la validación de una propiedad requiere que un modelo de vista tenga acceso a servicios, datos u otras API adicionales. Hay diferentes maneras de agregar validación personalizada a una propiedad, según el escenario y el nivel de flexibilidad que se requiere. Este es un ejemplo de cómo se puede usar el tipo [\[CustomValidationAttribute\]](#) para indicar que es necesario invocar un método específico para realizar una validación adicional de una propiedad:

C#

```
public class RegistrationForm : ObservableValidator
{
    private readonly IFancyService service;
```

```

public RegistrationForm(IFancyService service)
{
    this.service = service;
}

private string name;

[Required]
[MinLength(2)]
[MaxLength(100)]
[CustomValidation(typeof(RegistrationForm), nameof(ValidateName))]
public string Name
{
    get => this.name;
    set => SetProperty(ref this.name, value, true);
}

public static ValidationResult ValidateName(string name, ValidationContext
context)
{
    RegistrationForm instance = (RegistrationForm)context.ObjectInstance;
    bool isValid = instance.service.Validate(name);

    if (isValid)
    {
        return ValidationResult.Success;
    }

    return new("The name was not validated by the fancy service");
}
}

```

En este caso, tenemos un método estático `ValidateName` que realizará la validación en la propiedad `Name` a través de un servicio que se inserta en nuestro modelo de vista. Este método recibe el `name` valor de la propiedad y la instancia `ValidationContext` en uso, que contiene elementos como la instancia de modelo de vista, el nombre de la propiedad que se va a validar y, opcionalmente, un proveedor de servicios y algunas marcas personalizadas que podemos usar o establecer. En este caso, estamos recuperando la instancia `RegistrationForm` del contexto de validación y, desde allí, usamos el servicio insertado para validar la propiedad. Tenga en cuenta que esta validación se ejecutará junto a las especificadas en los demás atributos, por lo que podemos combinar métodos de validación personalizados y atributos de validación existentes, pero nos gusta.

Atributos de validación personalizados

Otra manera de realizar la validación personalizada es implementar un `[ValidationAttribute]` personalizado e insertar la lógica de validación en el método `IsValid` invalidado. Esto permite

una flexibilidad adicional en comparación con el enfoque descrito anteriormente, ya que facilita la reutilización del mismo atributo en varios lugares.

Supongamos que queríamos validar una propiedad en función de su valor relativo con respecto a otra propiedad en el mismo modelo de vista. El primer paso sería definir un `[GreaterThanAttribute]` personalizado, de la siguiente manera:

C#

```
public sealed class GreaterThanAttribute : ValidationAttribute
{
    public GreaterThanAttribute(string propertyName)
    {
        PropertyName = propertyName;
    }

    public string PropertyName { get; }

    protected override ValidationResult IsValid(object value, ValidationContext validationContext)
    {
        object
            instance = validationContext.ObjectInstance,
            otherValue =
instance.GetType().GetProperty(PropertyName).GetValue(instance);

        if (((IComparable)value).CompareTo(otherValue) > 0)
        {
            return ValidationResult.Success;
        }

        return new("The current value is smaller than the other one");
    }
}
```

A continuación, podemos agregar este atributo a nuestro modelo de vista:

C#

```
public class ComparableModel : ObservableValidator
{
    private int a;

    [Range(10, 100)]
    [GreaterThan(nameof(B))]
    public int A
    {
        get => this.a;
        set => SetProperty(ref this.a, value, true);
    }
}
```

```

private int b;

[Range(20, 80)]
public int B
{
    get => this.b;
    set
    {
        SetProperty(ref this.b, value, true);
        ValidateProperty(A, nameof(A));
    }
}
}

```

En este caso, tenemos dos propiedades numéricas que deben estar en un intervalo específico y con una relación específica entre sí (**A** debe ser mayor que **B**). Hemos agregado el nuevo `[GreaterThanAttribute]` sobre la primera propiedad y también hemos agregado una llamada a `ValidateProperty` en el establecedor para **B**, de modo que **A** se valide de nuevo cada vez que **B** cambie (ya que su estado de validación depende de él). Solo necesitamos estas dos líneas de código en nuestro modelo de vista para habilitar esta validación personalizada y también obtenemos la ventaja de tener un atributo de validación personalizado reutilizable que podría ser útil en otros modelos de vista de nuestra aplicación también. Este enfoque también ayuda con la modularización de código, ya que la lógica de validación ahora está completamente desacoplada de la propia definición del modelo de vista.

Ejemplos

- Consulte la [aplicación de ejemplo](#) (para varios marcos de interfaz de usuario) para ver el kit de herramientas de MVVM en acción.
- También puede encontrar más ejemplos en las [pruebas unitarias](#).

RelayCommand y RelayCommand<T>

Artículo • 07/11/2024

[RelayCommand](#) y [RelayCommand<T>](#) son implementaciones de `ICommand` que pueden exponer un método o un delegado a la vista. Estos tipos actúan como una manera de enlazar comandos entre el modelo de vista y los elementos de la interfaz de usuario.

API de la plataforma: [RelayCommand](#), [RelayCommand<T>](#), [IRelayCommand](#), [IRelayCommand<T>](#)

Cómo funcionan

`RelayCommand` y `RelayCommand<T>` tienen las siguientes características principales:

- Proporcionan una implementación base de la interfaz `ICommand`.
- También implementan la interfaz [IRelayCommand](#) (y [IRelayCommand<T>](#)), que expone un método `NotifyCanExecuteChanged` para generar el evento `CanExecuteChanged`.
- Exponen constructores que toman delegados como `Action` y `Func<T>`, lo que permite ajustar métodos estándar y expresiones lambda.

Uso de `ICommand`

A continuación se muestra cómo configurar un comando sencillo:

C#

```
public class MyViewModel : ObservableObject
{
    public MyViewModel()
    {
        IncrementCounterCommand = new RelayCommand(IncrementCounter);
    }

    private int counter;

    public int Counter
    {
        get => counter;
        private set => SetProperty(ref counter, value);
    }

    public ICommand IncrementCounterCommand { get; }

    private void IncrementCounter() => Counter++;
}
```

Y la interfaz de usuario relativa podría ser la siguiente (con XAML de WinUI):

XML

```
<Page
  x:Class="MyApp.Views.MyPage"
  xmlns:viewModels="using:MyApp.ViewModels">
  <Page.DataContext>
    <viewModels:MyViewModel x:Name="ViewModel"/>
  </Page.DataContext>

  <StackPanel Spacing="8">
    <TextBlock Text="{x:Bind ViewModel.Counter, Mode=OneWay}"/>
    <Button
      Content="Click me!"
      Command="{x:Bind ViewModel.IncrementCounterCommand}"/>
  </StackPanel>
</Page>
```

El objeto `Button` enlaza a `ICommand` en el modelo de vista, que ajusta el método `IncrementCounter` privado. El objeto `TextBlock` muestra el valor de la propiedad `Counter` y se actualiza cada vez que cambia el valor de la propiedad.

Ejemplos

- Consulte la [aplicación de ejemplo](#) (para varios marcos de interfaz de usuario) para ver el kit de herramientas de MVVM en acción.
- También puede encontrar más ejemplos en las [pruebas unitarias](#).

AsyncRelayCommand y AsyncRelayCommand<T>

Artículo • 07/11/2024

Los [AsyncRelayCommand](#) y [AsyncRelayCommand<T>](#) son `ICommand` implementaciones que amplían las funcionalidades que ofrece [RelayCommand](#), con compatibilidad con operaciones asincrónicas.

API de la plataforma: [AsyncRelayCommand](#), [AsyncRelayCommand<T>](#), [RelayCommand](#), [IAsyncRelayCommand](#), [IAsyncRelayCommand<T>](#)

Cómo funcionan

`AsyncRelayCommand` y `AsyncRelayCommand<T>` tienen las siguientes características principales:

- Amplían las funcionalidades de los comandos sincrónicos incluidos en la biblioteca, con compatibilidad con los delegados que devuelven `Task`.
- Pueden encapsular funciones asincrónicas con un parámetro `CancellationToken` adicional para admitir la cancelación y exponen una `CanBeCanceled` y `IsCancellationRequested` propiedades, así como un método `Cancel`.
- Exponen una propiedad `ExecutionTask` que se puede usar para supervisar el progreso de una operación pendiente y un `IsRunning` que se puede usar para comprobar cuándo se completa una operación. Esto resulta especialmente útil para enlazar un comando a elementos de la interfaz de usuario, como indicadores de carga.
- Implementan las interfaces [IAsyncRelayCommand](#) y [IAsyncRelayCommand<T>](#), lo que significa que el modelo de vista puede exponer fácilmente comandos con estos para reducir el acoplamiento estricto entre tipos. Por ejemplo, esto facilita la sustitución de un comando por una implementación personalizada que expone la misma superficie de API pública, si es necesario.

Trabajar con comandos asincrónicos

Imaginemos un escenario similar al descrito en el ejemplo `RelayCommand`, pero un comando que ejecuta una operación asincrónica:

```
C#  
  
public class MyViewModel : ObservableObject  
{  
    public MyViewModel()  
    {  
    }  
}
```



```

{
    DownloadTextCommand = new AsyncRelayCommand(DownloadText);
}

public IAsyncRelayCommand DownloadTextCommand { get; }

private Task<string> DownloadText()
{
    return WebService.LoadMyTextAsync();
}
}

```

Con el código de interfaz de usuario relacionado:

XML

```

<Page
    x:Class="MyApp.Views.MyPage"
    xmlns:viewModels="using:MyApp.ViewModels"
    xmlns:converters="using:Microsoft.Toolkit.Uwp.UI.Converters">
    <Page.DataContext>
        <viewModels:MyViewModel x:Name="ViewModel"/>
    </Page.DataContext>
    <Page.Resources>
        <converters:TaskResultConverter x:Key="TaskResultConverter"/>
    </Page.Resources>

    <StackPanel Spacing="8" xml:space="default">
        <TextBlock>
            <Run Text="Task status:"/>
            <Run Text="{x:Bind ViewModel.DownloadTextCommand.ExecutionTask.Status,
Mode=OneWay}"/>
            <LineBreak/>
            <Run Text="Result:"/>
            <Run Text="{x:Bind ViewModel.DownloadTextCommand.ExecutionTask,
Converter={StaticResource TaskResultConverter}, Mode=OneWay}"/>
        </TextBlock>
        <Button
            Content="Click me!"
            Command="{x:Bind ViewModel.DownloadTextCommand}"/>
        <ProgressRing
            HorizontalAlignment="Left"
            IsActive="{x:Bind ViewModel.DownloadTextCommand.IsRunning,
Mode=OneWay}"/>
    </StackPanel>
</Page>

```

Al hacer clic en el `Button`, se invoca el comando y se actualiza el `ExecutionTask`. Cuando se completa la operación, la propiedad genera una notificación que se refleja en la interfaz de usuario. En este caso, se muestran el estado de la tarea y el resultado actual de la tarea. Tenga en cuenta que para mostrar el resultado de la tarea, es necesario usar el método

`TaskExtensions.GetResultOrDefault`; esto proporciona acceso al resultado de una tarea que aún no se ha completado sin bloquear el subproceso (y posiblemente provocar un interbloqueo).

Ejemplos

- Consulte la [aplicación de ejemplo](#) (para varios marcos de interfaz de usuario) para ver el kit de herramientas de MVVM en acción.
- También puede encontrar más ejemplos en las [pruebas unitarias](#).

loc (inversión de control)

Artículo • 07/11/2024

Un patrón común que se puede usar para aumentar la modularidad en el código base de una aplicación mediante el patrón MVVM es usar alguna forma de inversión de control. Una de las soluciones más comunes, en particular, es usar la inserción de dependencias, que consiste en crear una serie de servicios que se inyectan en las clases del back-end (es decir, se pasan como parámetros a los constructores del modelo de vista). Esto permite que el código que usa estos servicios no dependa de los detalles de implementación de los mismos, y también facilita el intercambio de las implementaciones concretas de estos servicios. Este patrón también facilita que las características específicas de la plataforma estén disponibles para el código back-end, abstrayéndolas a través de un servicio que, a continuación, se inserta cuando sea necesario.

El kit de herramientas de MVVM no proporciona API integradas para facilitar la utilización de este patrón, puesto que ya existen bibliotecas dedicadas específicamente a ello, como el paquete `Microsoft.Extensions.DependencyInjection`, que proporciona un conjunto de API de inserción de dependencias completo y potente, y actúa como un `IServiceProvider` fácil de configurar y usar. La siguiente guía hará referencia a esta biblioteca y proporcionará una serie de ejemplos de cómo integrarla en aplicaciones mediante el patrón de MVVM.

API de la plataforma: [loc](#)

Configuración y resolución de servicios

El primer paso es declarar una instancia de `IServiceProvider` e inicializar todos los servicios necesarios, normalmente en el inicio. Por ejemplo, en UWP (pero también se puede usar una configuración similar en otros marcos):

C#

```
public sealed partial class App : Application
{
    public App()
    {
        Services = ConfigureServices();

        this.InitializeComponent();
    }

    /// <summary>
    /// Gets the current <see cref="App"/> instance in use
    /// </summary>
    public new static App Current => (App)Application.Current;

    /// <summary>
```

```

    /// Gets the <see cref="IServiceProvider"/> instance to resolve application
    services.
    /// </summary>
    public IServiceProvider Services { get; }

    /// <summary>
    /// Configures the services for the application.
    /// </summary>
    private static IServiceProvider ConfigureServices()
    {
        var services = new ServiceCollection();

        services.AddSingleton<IFilesService, FilesService>();
        services.AddSingleton<ISettingsService, SettingsService>();
        services.AddSingleton<IClipboardService, ClipboardService>();
        services.AddSingleton<IShareService, ShareService>();
        services.AddSingleton<IEmailService, EmailService>();

        return services.BuildServiceProvider();
    }
}

```

Aquí se inicializa la propiedad `Services` en el inicio y se registran todos los servicios de aplicación y los modelos de vista. También hay una nueva propiedad `Current` que se puede usar para acceder fácilmente a la propiedad `Services` desde otras vistas de la aplicación. Por ejemplo:

C#

```

IFilesService filesService = App.Current.Services.GetService<IFilesService>();

// Use the files service here...

```

El aspecto clave aquí es que cada servicio puede perfectamente estar usando API específicas de la plataforma, pero como todas ellas están abstraídas a través de la interfaz que está usando nuestro código, no necesitamos preocuparnos por ellas cuando solo estamos resolviendo una instancia y usándola para realizar operaciones.

Inserción de constructores

Una potente característica disponible es la "inserción de constructores", lo que significa que el proveedor de servicios de DI es capaz de resolver automáticamente las dependencias indirectas entre los servicios registrados al crear instancias del tipo que se está solicitando. Considere el siguiente servicio:

C#

```

public class FileLogger : IFileLogger
{
    private readonly IFilesService FileService;
    private readonly IConsoleService ConsoleService;

    public FileLogger(
        IFilesService fileService,
        IConsoleService consoleService)
    {
        FileService = fileService;
        ConsoleService = consoleService;
    }

    // Methods for the IFileLogger interface here...
}

```

Aquí tenemos un tipo `FileLogger` que implementa la interfaz `IFileLogger` y requiere instancias de `IFilesService` y `IConsoleService`. La inserción de constructores significa que el proveedor de servicios de DIs recopilará automáticamente todos los servicios necesarios, de la siguiente manera:

```

C#

/// <summary>
/// Configures the services for the application.
/// </summary>
private static IServiceProvider ConfigureServices()
{
    var services = new ServiceCollection();

    services.AddSingleton<IFilesService, FilesService>();
    services.AddSingleton<IConsoleService, ConsoleService>();
    services.AddSingleton<IFileLogger, FileLogger>();

    return services.BuildServiceProvider();
}

// Retrieve a logger service with constructor injection
IFileLogger fileLogger = App.Current.Services.GetService<IFileLogger>();

```

El proveedor de servicios de DI comprobará automáticamente si todos los servicios necesarios están registrados, los recuperará e invocará el constructor para el tipo concreto registrado `IFileLogger`, para obtener la instancia que se va a devolver.

¿Qué ocurre con los modelos de vista?

Un proveedor de servicios tiene "servicio" en su nombre, pero realmente se puede usar para resolver instancias de cualquier clase, incluidos los modelos de vista. Se siguen aplicando los mismos conceptos explicados anteriormente, incluida la inserción de constructores. Imaginemos que tenemos un tipo `ContactsViewModel`, usando una instancia de `IServiceProvider` y otra de `IService` a través de su constructor. Podríamos tener un método `ConfigureServices` similar al siguiente:

C#

```
/// <summary>
/// Configures the services for the application.
/// </summary>
private static IServiceProvider ConfigureServices()
{
    var services = new ServiceCollection();

    // Services
    services.AddSingleton<IContactsService, ContactsService>();
    services.AddSingleton<IPhoneService, PhoneService>();

    // Viewmodels
    services.AddTransient<ContactsViewModel>();

    return services.BuildServiceProvider();
}
```

Y, después, en nuestra `ContactsView`, asignaríamos el contexto de datos de la siguiente manera:

C#

```
public ContactsView()
{
    this.InitializeComponent();
    this.DataContext = App.Current.Services.GetService<ContactsViewModel>();
}
```

Más documentos

Para más información sobre `Microsoft.Extensions.DependencyInjection`, vaya [aquí](#).

Ejemplos

- Consulte la [aplicación de ejemplo](#) (para varios marcos de interfaz de usuario) para ver el kit de herramientas de MVVM en acción.

- También puede encontrar más ejemplos en las [pruebas unitarias](#).

Mensajero

Artículo • 07/11/2024

La interfaz `IMessenger` es un contrato para los tipos que se pueden usar para intercambiar mensajes entre distintos objetos. Esto puede ser útil para desacoplar diferentes módulos de una aplicación sin tener que mantener referencias seguras a los tipos a los que se hace referencia. También es posible enviar mensajes a canales específicos, identificados de forma única por un token, y tener mensajeros diferentes en diferentes secciones de una aplicación. El kit de herramientas de MVVM proporciona dos implementaciones de fábrica:

`WeakReferenceMessenger` y `StrongReferenceMessenger`: la primera usa referencias débiles internamente, ofreciendo administración automática de memoria para destinatarios, mientras que esta última usa referencias seguras y requiere que los desarrolladores cancelen manualmente la suscripción de sus destinatarios cuando ya no sean necesarios (más detalles sobre cómo anular el registro de controladores de mensajes se pueden encontrar a continuación), pero a cambio de eso ofrece un mejor rendimiento y mucho menos uso de memoria.

API de la plataforma: `IMessenger`, `WeakReferenceMessenger`, `StrongReferenceMessenger`, `IRecipient<TMessage>`, `MessageHandler<TRecipient, TMessage>`, `ObservableRecipient`, `RequestMessage<T>`, `AsyncRequestMessage<T>`, `CollectionRequestMessage<T>`, `AsyncCollectionRequestMessage<T>`.

Funcionamiento

Los tipos que implementan `IMessenger` son responsables de mantener vínculos entre destinatarios (receptores de mensajes) y sus tipos de mensajes registrados, con controladores de mensajes relativos. Cualquier objeto se puede registrar como destinatario para un tipo de mensaje determinado mediante un controlador de mensajes, que se invocará siempre que se use la instancia `IMessenger` para enviar un mensaje de ese tipo. También es posible enviar mensajes a través de canales de comunicación específicos (cada uno identificado por un token único), de modo que varios módulos puedan intercambiar mensajes del mismo tipo sin causar conflictos. Los mensajes enviados sin un token usan el canal compartido predeterminado.

Hay dos maneras de realizar el registro de mensajes: a través de la interfaz `IRecipient<TMessage>` o mediante un delegado `MessageHandler<TRecipient, TMessage>` que actúa como controlador de mensajes. La primera permite registrar todos los controladores con una sola llamada a la extensión `RegisterAll`, que registra automáticamente los destinatarios de todos los controladores de mensajes declarados, mientras que este último resulta útil cuando necesita más flexibilidad o cuando desea usar una expresión lambda simple como controlador de mensajes.

Tanto `WeakReferenceMessenger` y `StrongReferenceMessenger` también exponen una propiedad `Default` que ofrece una implementación segura para subprocesos integrada en el paquete. También es posible crear varias instancias de mensajes si es necesario, por ejemplo, si se inserta otra con un proveedor de servicios de inserción de dependencias en un módulo diferente de la aplicación (por ejemplo, varias ventanas que se ejecutan en el mismo proceso).

❗ Nota

Dado que el tipo `WeakReferenceMessenger` es más sencillo de usar y coincide con el comportamiento del tipo mensaje de la biblioteca `MvvmLight`, es el tipo predeterminado que usa el tipo `ObservableRecipient` en el kit de herramientas de MVVM. El `StrongReferenceType` todavía se puede usar, pasando una instancia al constructor de esa clase.

Envío y recepción de mensajes

Tenga en cuenta lo siguiente.

C#

```
// Create a message
public class LoggedInUserChangedMessage : ValueChangedMessage<User>
{
    public LoggedInUserChangedMessage(User user) : base(user)
    {
    }
}

// Register a message in some module
WeakReferenceMessenger.Default.Register<LoggedInUserChangedMessage>(this, (r, m)
=>
{
    // Handle the message here, with r being the recipient and m being the
    // input message. Using the recipient passed as input makes it so that
    // the lambda expression doesn't capture "this", improving performance.
}));

// Send a message from some other module
WeakReferenceMessenger.Default.Send(new LoggedInUserChangedMessage(user));
```

Imaginemos que este tipo de mensaje se usa en una aplicación de mensajería sencilla, que muestra un encabezado con el nombre de usuario y la imagen de perfil del usuario registrado actualmente, un panel con una lista de conversaciones y otro panel con mensajes de la conversación actual, si se selecciona uno. Supongamos que estas tres secciones son

compatibles con los tipos `HeaderViewModel`, `ConversationsListViewModel` y `ConversationViewModel` respectivamente. En este escenario, el `HeaderViewModel` puede enviar el mensaje `LoggedInUserChangedMessage` una vez completada una operación de inicio de sesión, y ambos modelos de vista pueden registrar controladores para él. Por ejemplo, `ConversationsListViewModel` cargará la lista de conversaciones para el nuevo usuario y `ConversationViewModel` simplemente cerrará la conversación actual, si hay una presente.

La instancia `IMessenger` se encarga de entregar mensajes a todos los destinatarios registrados. Tenga en cuenta que un destinatario puede suscribirse a mensajes de un tipo específico. Tenga en cuenta que los tipos de mensajes heredados no están registrados en las implementaciones `IMessenger` predeterminadas proporcionadas por el kit de herramientas de MVVM.

Cuando ya no se necesite un destinatario, debe anular el registro para que deje de recibir mensajes. Puede anular el registro por tipo de mensaje, por token de registro o por destinatario:

C#

```
// Unregisters the recipient from a message type
WeakReferenceMessenger.Default.Unregister<LoggedInUserChangedMessage>(this);

// Unregisters the recipient from a message type in a specified channel
WeakReferenceMessenger.Default.Unregister<LoggedInUserChangedMessage, int>(this,
42);

// Unregister the recipient from all messages, across all channels
WeakReferenceMessenger.Default.UnregisterAll(this);
```

⚠ Advertencia

Como se mencionó antes, esto no es estrictamente necesario cuando se usa el tipo `WeakReferenceMessenger`, ya que usa referencias débiles para realizar un seguimiento de los destinatarios, lo que significa que los destinatarios no usados seguirán siendo aptos para la recolección de elementos no utilizados aunque todavía tengan controladores de mensajes activos. Sin embargo, sigue siendo recomendable cancelar su suscripción para mejorar los rendimientos. Por otro lado, la implementación `StrongReferenceMessenger` usa referencias seguras para realizar un seguimiento de los destinatarios registrados. Esto se hace por motivos de rendimiento y significa que cada destinatario registrado debe anularse manualmente para evitar pérdidas de memoria. Es decir, siempre que se registre un destinatario, la instancia `StrongReferenceMessenger` en uso mantendrá una referencia activa a él, lo que impedirá que el recolector de elementos no utilizados pueda recopilar esa instancia. Puede controlar esto manualmente o puede heredar de `ObservableRecipient`, que de forma predeterminada se encarga de quitar todos los

registros de mensajes para el destinatario cuando se desactiva (vea los documentos en `ObservableRecipient` para obtener más información sobre esto).

También es posible usar la interfaz `IRecipient<TMessage>` para registrar controladores de mensajes. En este caso, cada destinatario deberá implementar la interfaz para un tipo de mensaje determinado y proporcionar un método `Receive(TMessage)` que se invocará al recibir mensajes, de la siguiente manera:

```
C#

// Create a message
public class MyRecipient : IRecipient<LoggedInUserChangedMessage>
{
    public void Receive(LoggedInUserChangedMessage message)
    {
        // Handle the message here...
    }
}

// Register that specific message...
WeakReferenceMessenger.Default.Register<LoggedInUserChangedMessage>(this);

// ...or alternatively, register all declared handlers
WeakReferenceMessenger.Default.RegisterAll(this);

// Send a message from some other module
WeakReferenceMessenger.Default.Send(new LoggedInUserChangedMessage(user));
```

Uso de mensajes de solicitud

Otra característica útil de las instancias de mensajes es que también se pueden usar para solicitar valores de un módulo a otro. Para ello, el paquete incluye una clase base

`RequestMessage<T>`, que se puede usar de la siguiente manera:

```
C#

// Create a message
public class LoggedInUserRequestMessage : RequestMessage<User>
{
}

// Register the receiver in a module
WeakReferenceMessenger.Default.Register<MyViewModel, LoggedInUserRequestMessage>
(this, (r, m) =>
{
    // Assume that "CurrentUser" is a private member in our viewmodel.
    // As before, we're accessing it through the recipient passed as
```

```
// input to the handler, to avoid capturing "this" in the delegate.
m.Reply(r.CurrentUser);
});

// Request the value from another module
User user = WeakReferenceMessenger.Default.Send<LoggedInUserRequestMessage>();
```

La clase `RequestMessage<T>` incluye un convertidor implícito que hace posible la conversión de un `LoggedInUserRequestMessage` a su objeto `User` contenido. Esto también comprobará que se ha recibido una respuesta para el mensaje y producirá una excepción si no es así. También es posible enviar mensajes de solicitud sin esta garantía de respuesta obligatoria: solo tiene que almacenar el mensaje devuelto en una variable local y, a continuación, comprobar manualmente si hay un valor de respuesta disponible o no. Si no lo hace, no se desencadenará la excepción automática si no se recibe una respuesta cuando se devuelve el método `Send`.

El mismo espacio de nombres también incluye el mensaje de solicitudes base para otros escenarios: `AsyncRequestMessage<T>`, `CollectionRequestMessage<T>` y `AsyncCollectionRequestMessage<T>`. Aquí se muestra cómo puede usar un mensaje de solicitud asincrónica:

C#

```
// Create a message
public class LoggedInUserRequestMessage : AsyncRequestMessage<User>
{
}

// Register the receiver in a module
WeakReferenceMessenger.Default.Register<MyViewModel, LoggedInUserRequestMessage>
(this, (r, m) =>
{
    m.Reply(r.GetCurrentUserAsync()); // We're replying with a Task<User>
});

// Request the value from another module (we can directly await on the request)
User user = await WeakReferenceMessenger.Default.Send<LoggedInUserRequestMessage>
();
```

Ejemplos

- Consulte la [aplicación de ejemplo](#) (para varios marcos de interfaz de usuario) para ver el kit de herramientas de MVVM en acción.
- También puede encontrar más ejemplos en las [pruebas unitarias](#).

Poner las cosas en orden

Ahora que hemos descrito todos los distintos componentes que están disponibles en el paquete `CommunityToolkit.Mvvm`, podemos ver un ejemplo práctico de todos ellos en su conjunto para crear un único ejemplo más grande. En este caso, queremos crear un explorador de Reddit muy simple y minimalista para un número selecto de subreddits.

¿Qué queremos compilar?

Comencemos por esquematizar exactamente lo que queremos compilar:

- Un navegador mínimo de Reddit compuesto por dos "widgets": uno que muestra las publicaciones de un subreddit y otro que muestra la publicación seleccionada en ese momento. Los dos widgets deben ser independientes y sin referencias fuertes entre sí.
- Queremos que los usuarios puedan seleccionar un subreddit de una lista de opciones disponibles y queremos guardar el subreddit seleccionado como configuración y cargarlo la próxima vez que se cargue el ejemplo.
- Queremos que el widget subreddit también ofrezca un botón de actualización para volver a cargar el subreddit actual.
- Para los fines de este ejemplo, no es necesario poder controlar todos los tipos de publicación posibles. Solo asignaremos un texto de ejemplo a todas las publicaciones cargadas y lo mostraremos directamente para simplificar las cosas.

Configuración de los modelos de vista

Empecemos con el modelo de vista que hará funcionar el widget subreddit y repasemos las herramientas que necesitamos:

- **Comandos:** necesitamos la vista para poder solicitar el modelo de vista para volver a cargar la lista actual de publicaciones desde el subreddit seleccionado. Podemos usar el tipo `AsyncRelayCommand` para encapsular un método privado que capturará las publicaciones de Reddit. Aquí se expone el comando a través de la interfaz `IAAsyncRelayCommand` para evitar referencias fuertes al tipo de comando exacto que estamos usando. Esto también nos permitirá cambiar potencialmente el tipo de comando en el futuro sin tener que preocuparnos por ningún componente de interfaz de usuario que dependa de ese tipo específico que se use.
- **Propiedades:** necesitamos exponer una serie de valores a la interfaz de usuario, podemos hacerlo con propiedades observables si son valores que pretendemos reemplazar por completo o con propiedades que son observables por sí mismas (por ejemplo, `ObservableCollection<T>`). En este caso, tenemos:

- `ObservableCollection<object> Posts`, que es la lista observable de publicaciones cargadas. Aquí solo usamos `object` como marcador de posición, ya que aún no hemos creado un modelo para representar publicaciones. Podemos reemplazar esto más adelante.
- `ReadOnlyList<string> Subreddits`, que es una lista de solo lectura con los nombres de los subreddits entre los que se permite a los usuarios elegir. Esta propiedad nunca se actualiza, por lo que tampoco es necesario que sea observable.
- `string SelectedSubreddit`, que es el subreddit seleccionado actualmente. Esta propiedad debe enlazarse a la interfaz de usuario, ya que se usará tanto para indicar el último subreddit seleccionado cuando se cargue el ejemplo y para manipularse directamente desde la interfaz de usuario a medida que el usuario cambia la selección. Aquí se usa el método `SetProperty` de la clase `ObservableObject`.
- `object SelectedPost`, que es la publicación seleccionada actualmente. En este caso, estamos usando el método `SetProperty` de la clase `ObservableRecipient` para indicar que también queremos difundir notificaciones cuando cambia esta propiedad. Esto se hace para poder notificar al widget de publicación que se cambia la selección de publicación actual.
- **Métodos:** solo necesitamos un método privado `LoadPostsAsync` que se ajustará mediante nuestro comando asíncrono y que contendrá la lógica para cargar publicaciones desde el subreddit seleccionado.

Este es el modelo de vista hasta ahora:

C#

```
public sealed class SubredditWidgetViewModel : ObservableRecipient
{
    /// <summary>
    /// Creates a new <see cref="SubredditWidgetViewModel"/> instance.
    /// </summary>
    public SubredditWidgetViewModel()
    {
        LoadPostsCommand = new AsyncRelayCommand(LoadPostsAsync);
    }

    /// <summary>
    /// Gets the <see cref="IAsyncRelayCommand"/> instance responsible for loading
    posts.
    /// </summary>
    public IAsyncRelayCommand LoadPostsCommand { get; }

    /// <summary>
    /// Gets the collection of loaded posts.
    /// </summary>
    public ObservableCollection<object> Posts { get; } = new
    ObservableCollection<object>();
```

```

/// <summary>
/// Gets the collection of available subreddits to pick from.
/// </summary>
public IReadOnlyList<string> Subreddits { get; } = new[]
{
    "microsoft",
    "windows",
    "surface",
    "windowsphone",
    "dotnet",
    "csharp"
};

private string selectedSubreddit;

/// <summary>
/// Gets or sets the currently selected subreddit.
/// </summary>
public string SelectedSubreddit
{
    get => selectedSubreddit;
    set => SetProperty(ref selectedSubreddit, value);
}

private object selectedPost;

/// <summary>
/// Gets or sets the currently selected subreddit.
/// </summary>
public object SelectedPost
{
    get => selectedPost;
    set => SetProperty(ref selectedPost, value, true);
}

/// <summary>
/// Loads the posts from a specified subreddit.
/// </summary>
private async Task LoadPostsAsync()
{
    // TODO...
}
}

```

Ahora echemos un vistazo a lo que necesitamos para el modelo de vista del widget de publicación. Este será un modelo de vista mucho más sencillo, ya que realmente solo necesita exponer una propiedad `Post` con la publicación seleccionada actualmente y para recibir mensajes de difusión del widget subreddit para actualizar la propiedad `Post`. Puede tener un aspecto similar al siguiente:

```

public sealed class PostWidgetViewModel : ObservableRecipient,
IRecipient<PropertyChangedMessage<object>>
{
    private object post;

    /// <summary>
    /// Gets the currently selected post, if any.
    /// </summary>
    public object Post
    {
        get => post;
        private set => SetProperty(ref post, value);
    }

    /// <inheritdoc/>
    public void Receive(PropertyChangedMessage<object> message)
    {
        if (message.Sender.GetType() == typeof(SubredditWidgetViewModel) &&
            message.PropertyName == nameof(SubredditWidgetViewModel.SelectedPost))
        {
            Post = message.NewValue;
        }
    }
}

```

En este caso, estamos usando la interfaz `IRecipient<TMessage>` para declarar los mensajes que queremos que reciba nuestro modelo de vista. La clase `ObservableRecipient` agregará automáticamente los controladores de los mensajes declarados cuando la propiedad `IsActive` esté establecida en `true`. Tenga en cuenta que no es obligatorio usar este enfoque y también es posible registrar manualmente cada controlador de mensajes, como se indica a continuación:

C#

```

public sealed class PostWidgetViewModel : ObservableRecipient
{
    protected override void OnActivated()
    {
        // We use a method group here, but a lambda expression is also valid
        Messenger.Register<PostWidgetViewModel, PropertyChangedMessage<object>>
(this, (r, m) => r.Receive(m));
    }

    /// <inheritdoc/>
    public void Receive(PropertyChangedMessage<object> message)
    {
        if (message.Sender.GetType() == typeof(SubredditWidgetViewModel) &&
            message.PropertyName == nameof(SubredditWidgetViewModel.SelectedPost))
        {
            Post = message.NewValue;
        }
    }
}

```



```
}  
}
```

Ahora tenemos un borrador de nuestros modelos de vista listos y podemos empezar a examinar los servicios que necesitamos.

Creación del servicio de configuración

⚠ Nota:

El ejemplo se crea mediante el patrón de inserción de dependencias, que es el enfoque recomendado para lidiar con los servicios en modelos de vista. También es posible usar otros patrones, como el patrón de localizador de servicios, pero el kit de herramientas de MVVM no ofrece API integradas para habilitarlo.

Dado que queremos que algunas de nuestras propiedades se guarden y conserven, necesitamos una manera para que los modelos de vista puedan interactuar con la configuración de la aplicación. Sin embargo, no deberíamos usar API específicas de la plataforma directamente en nuestros modelos de vista, ya que eso nos impediría tener todos nuestros modelos de vista en un proyecto portable de .NET Standard. Podemos resolver este problema mediante servicios y las API de la biblioteca

`Microsoft.Extensions.DependencyInjection` para configurar la instancia `IServiceProvider` de la aplicación. La idea es escribir interfaces que representen toda la superficie de API que necesitamos y, a continuación, implementar tipos específicos de la plataforma que implementan esta interfaz en todos los destinos de la aplicación. Los modelos de vista solo interactuarán con las interfaces, por lo que no tendrán ninguna referencia fuerte a ningún tipo específico de la plataforma.

Esta es una interfaz sencilla para un servicio de configuración:

C#

```
public interface ISettingsService  
{  
    /// <summary>  
    /// Assigns a value to a settings key.  
    /// </summary>  
    /// <typeparam name="T">The type of the object bound to the key.</typeparam>  
    /// <param name="key">The key to check.</param>  
    /// <param name="value">The value to assign to the setting key.</param>  
    void SetValue<T>(string key, T value);  
  
    /// <summary>  
    /// Reads a value from the current <see cref="IServiceProvider"/> instance and
```

```

returns its casting in the right type.
    /// </summary>
    /// <typeparam name="T">The type of the object to retrieve.</typeparam>
    /// <param name="key">The key associated to the requested object.</param>
    [Pure]
    T GetValue<T>(string key);
}

```

Podemos suponer que los tipos específicos de la plataforma que implementan esta interfaz se encargarán de tratar con toda la lógica necesaria para serializar realmente la configuración, almacenarla en el disco y, a continuación, leerla de nuevo. Ahora podemos usar este servicio en `SubredditWidgetViewModel`, para que la propiedad `SelectedSubreddit` sea persistente:

C#

```

/// <summary>
/// Gets the <see cref="ISettingsService"/> instance to use.
/// </summary>
private readonly ISettingsService SettingsService;

/// <summary>
/// Creates a new <see cref="SubredditWidgetViewModel"/> instance.
/// </summary>
public SubredditWidgetViewModel(ISettingsService settingsService)
{
    SettingsService = settingsService;

    selectedSubreddit = settingsService.GetValue<string>(nameof(SelectedSubreddit))
?? Subreddits[0];
}

private string selectedSubreddit;

/// <summary>
/// Gets or sets the currently selected subreddit.
/// </summary>
public string SelectedSubreddit
{
    get => selectedSubreddit;
    set
    {
        SetProperty(ref selectedSubreddit, value);

        SettingsService.SetValue(nameof(SelectedSubreddit), value);
    }
}

```

Aquí se usa la inserción de dependencias y la inserción de constructores, como se mencionó anteriormente. Hemos declarado un campo `ISettingsService SettingsService` que simplemente almacena nuestro servicio de configuración (que estamos recibiendo como parámetro en el constructor de modelo de vista) y, a continuación, estamos inicializando la

propiedad `SelectedSubreddit` en el constructor, ya sea mediante el valor anterior o solo el primer subreddit disponible. A continuación, también hemos modificado el establecedor `SelectedSubreddit`, de modo que también usará el servicio de configuración para guardar el nuevo valor en el disco.

Magnífico. Ahora solo tenemos que escribir una versión específica de la plataforma de este servicio, esta vez directamente dentro de uno de nuestros proyectos de aplicación. Este es el aspecto que podría tener ese servicio en UWP:

C#

```
public sealed class SettingsService : ISettingsService
{
    /// <summary>
    /// The <see cref="IPropertySet"/> with the settings targeted by the current
    /// instance.
    /// </summary>
    private readonly IPropertySet SettingsStorage =
        ApplicationData.Current.LocalSettings.Values;

    /// <inheritdoc/>
    public void SetValue<T>(string key, T value)
    {
        if (!SettingsStorage.ContainsKey(key)) SettingsStorage.Add(key, value);
        else SettingsStorage[key] = value;
    }

    /// <inheritdoc/>
    public T GetValue<T>(string key)
    {
        if (SettingsStorage.TryGetValue(key, out object value))
        {
            return (T)value;
        }

        return default;
    }
}
```

La última parte del rompecabezas es insertar este servicio específico de la plataforma en nuestra instancia del proveedor de servicios. Podemos hacerlo en el inicio, de la siguiente manera:

C#

```
/// <summary>
/// Gets the <see cref="IServiceProvider"/> instance to resolve application
/// services.
/// </summary>
public IServiceProvider Services { get; }
```

```

/// <summary>
/// Configures the services for the application.
/// </summary>
private static IServiceProvider ConfigureServices()
{
    var services = new ServiceCollection();

    services.AddSingleton<ISettingsService, SettingsService>();
    services.AddTransient<PostWidgetViewModel>();

    return services.BuildServiceProvider();
}

```

Esto registrará una instancia singleton de `SettingsService` como un tipo que implementa `ISettingsService`. También estamos registrando `PostWidgetViewModel` como un servicio transitorio, lo que significa que cada vez que recuperamos una instancia, será una nueva (puede imaginarse que esto es útil si desea tener varios widgets de publicación independientes). Esto significa que cada vez que se resuelve una instancia `ISettingsService` mientras la aplicación en uso es la UWP, recibirá una instancia `SettingsService`, que usará las API de UWP en segundo plano para manipular la configuración. lo que resulta ideal.

Creación del servicio de Reddit

El último componente del back-end que falta es un servicio que puede usar las API de REST de Reddit para capturar las publicaciones de los subreddits que nos interesan. Para compilarlo, vamos a usar [refit](#), que es una biblioteca para crear fácilmente servicios con seguridad de tipos para interactuar con las API de REST. Como antes, es necesario definir la interfaz con todas las API que implementará nuestro servicio, de la siguiente manera:

```

C#

public interface IRedditService
{
    /// <summary>
    /// Get a list of posts from a given subreddit
    /// </summary>
    /// <param name="subreddit">The subreddit name.</param>
    [Get("/r/{subreddit}/.json")]
    Task<PostsQueryResponse> GetSubredditPostsAsync(string subreddit);
}

```

Que `PostsQueryResponse` es un modelo que hemos escrito que asigna la respuesta JSON para esa API. La estructura exacta de esa clase no es importante, basta con decir que contiene una colección de elementos `Post`, que son modelos simples que representan nuestras publicaciones, así:

C#

```
public class Post
{
    /// <summary>
    /// Gets or sets the title of the post.
    /// </summary>
    public string Title { get; set; }

    /// <summary>
    /// Gets or sets the URL to the post thumbnail, if present.
    /// </summary>
    public string Thumbnail { get; set; }

    /// <summary>
    /// Gets the text of the post.
    /// </summary>
    public string SelfText { get; }
}
```

Una vez que tenemos nuestro servicio y nuestros modelos, podemos conectarlos a nuestros modelos de vista para completar nuestro back-end. Al hacerlo, también podemos reemplazar esos marcadores de posición `object` por el tipo `Post` que hemos definido:

C#

```
public sealed class SubredditWidgetViewModel : ObservableRecipient
{
    /// <summary>
    /// Gets the <see cref="IRedditService"/> instance to use.
    /// </summary>
    private readonly IR redditService =
        Ioc.Default.GetRequiredService<IR redditService>();

    /// <summary>
    /// Loads the posts from a specified subreddit.
    /// </summary>
    private async Task LoadPostsAsync()
    {
        var response = await
            redditService.GetSubredditPostsAsync(SelectedSubreddit);

        Posts.Clear();

        foreach (var item in response.Data.Items)
        {
            Posts.Add(item.Data);
        }
    }
}
```

Hemos agregado un nuevo campo `IRedditService` para almacenar nuestro servicio, al igual que hicimos para el servicio de configuración, y hemos implementado nuestro método `LoadPostsAsync`, que anteriormente estaba vacío.

La última pieza que falta ahora es insertar el servicio real en nuestro proveedor de servicios. La gran diferencia en este caso es que al usar `refit` no es necesario implementar el servicio en absoluto. La biblioteca creará automáticamente un tipo que implementa el servicio para nosotros, en segundo plano. Por lo tanto, solo necesitamos obtener una instancia `IRedditService` e insertarla directamente, de la siguiente manera:

C#

```
/// <summary>
/// Configures the services for the application.
/// </summary>
private static IServiceProvider ConfigureServices()
{
    var services = new ServiceCollection();

    services.AddSingleton<ISettingsService, SettingsService>();
    services.AddSingleton(RestService.For<IRedditService>
("https://www.reddit.com/"));
    services.AddTransient<PostWidgetViewModel>();

    return services.BuildServiceProvider();
}
```

¡Y eso es todo lo que necesitamos hacer! Ahora tenemos todo nuestro back-end listo para usarlo, incluidos dos servicios personalizados que creamos específicamente para esta aplicación. 🎉

Creación de la interfaz de usuario

Ahora que se ha completado todo el back-end, podemos escribir la interfaz de usuario para nuestros widgets. Tenga en cuenta cómo el uso del patrón MVVM nos permite centrarnos exclusivamente en la lógica de negocios al principio, sin tener que escribir ningún código relacionado con la interfaz de usuario hasta ahora. Aquí quitaremos todo el código de la interfaz de usuario que no interactúe con nuestros modelos de vista, por motivos de simplicidad, y examinaremos cada control diferente uno por uno. El código fuente completo se puede encontrar en la aplicación de ejemplo.

Antes de pasar por los distintos controles, aquí se muestra cómo podemos resolver modelos de vista para todas las vistas diferentes de nuestra aplicación (por ejemplo, la `PostWidgetView`):

C#

```
public PostWidgetView()  
{  
    this.InitializeComponent();  
    this.DataContext = App.Current.Services.GetService<PostWidgetViewModel>();  
}  
  
public PostWidgetViewModel ViewModel => (PostWidgetViewModel)DataContext;
```

Estamos usando nuestra instancia `IServiceProvider` para resolver el objeto `PostWidgetViewModel` que necesitamos, que se asigna a la propiedad de contexto de datos. También estamos creando una propiedad `ViewModel` fuertemente tipada que simplemente convierte el contexto de datos al tipo de modelo de vista correcto; esto es necesario para habilitar `x:Bind` en el código XAML.

Comencemos con el widget subreddit, que incluye un `ComboBox` para seleccionar un subreddit, un `Button` para actualizar la fuente, un `ListView` para mostrar publicaciones y un `ProgressBar` para indicar cuándo se carga la fuente. Asumiremos que la propiedad `ViewModel` representa una instancia del modelo de vista que hemos descrito antes: esto se puede declarar en XAML o directamente en el código subyacente.

Selector de subreddit:

XML

```
<ComboBox  
    ItemsSource="{x:Bind ViewModel.Subreddits}"  
    SelectedItem="{x:Bind ViewModel.SelectedSubreddit, Mode=TwoWay}">  
    <interactivity:Interaction.Behaviors>  
        <core:EventTriggerBehavior EventName="SelectionChanged">  
            <core:InvokeCommandAction Command="{x:Bind  
ViewModel.LoadPostsCommand}"/>  
        </core:EventTriggerBehavior>  
    </interactivity:Interaction.Behaviors>  
</ComboBox>
```

Aquí se enlaza el origen a la propiedad `Subreddits` y el elemento seleccionado a la propiedad `SelectedSubreddit`. Observe cómo la propiedad `Subreddits` solo se enlaza una vez, ya que la propia colección envía notificaciones de cambio, mientras la propiedad `SelectedSubreddit` está enlazada con el modo `TwoWay`, ya que lo necesitamos para poder cargar el valor que recuperamos de nuestra configuración, así como actualizar la propiedad en el modelo de vista cuando el usuario cambia la selección. Además, usamos un comportamiento XAML para invocar nuestro comando cada vez que cambia la selección.

Botón de Actualizar:

XML

```
<Button Command="{x:Bind ViewModel.LoadPostsCommand}"/>
```

Este componente es extremadamente sencillo, solo estamos enlazando nuestro comando personalizado a la propiedad `Command` del botón, de modo que el comando se invocará siempre que el usuario haga clic en él.

Lista de publicaciones:

XML

```
<ListView
    ItemsSource="{x:Bind ViewModel.Posts}"
    SelectedItem="{x:Bind ViewModel.SelectedPost, Mode=TwoWay}">
    <ListView.ItemTemplate>
        <DataTemplate x:DataType="models:Post">
            <Grid>
                <TextBlock Text="{x:Bind Title}"/>
                <controls:ImageEx Source="{x:Bind Thumbnail}"/>
            </Grid>
        </DataTemplate>
    </ListView.ItemTemplate>
</ListView>
```

Aquí tenemos un `ListView` enlazando el origen y la selección a nuestra propiedad viewmodel y también una plantilla que se usa para mostrar cada publicación disponible. Estamos usando `x:DataType` para habilitar `x:Bind` en nuestra plantilla y tenemos dos controles que se enlazan directamente a las propiedades `Title` y `Thumbnail` de nuestra publicación.

Barra de carga:

XML

```
<ProgressBar Visibility="{x:Bind ViewModel.LoadPostsCommand.IsRunning,
Mode=OneWay}"/>
```

Aquí estamos enlazando a la propiedad `IsRunning`, que forma parte de la interfaz `IAsyncRelayCommand`. El tipo `AsyncRelayCommand` se encargará de generar notificaciones para esa propiedad cada vez que se inicie o complete la operación asíncronica para ese comando.

La última pieza que falta es la interfaz de usuario del widget de publicación. Como antes, hemos quitado todo el código relacionado con la interfaz de usuario que no era necesario para interactuar con los modelos de vista, por motivos de simplicidad. El código fuente completo está disponible en la aplicación de ejemplo.

XML

```
<Grid>

    <!--Header-->
    <Grid>
        <TextBlock Text="{x:Bind ViewModel.Post.Title, Mode=OneWay}"/>
        <controls:ImageEx Source="{x:Bind ViewModel.Post.Thumbnail, Mode=OneWay}"/>
    </Grid>

    <!--Content-->
    <ScrollView>
        <TextBlock Text="{x:Bind ViewModel.Post.SelfText, Mode=OneWay}"/>
    </ScrollView>
</Grid>
```

Aquí solo tenemos un encabezado, con un `TextBlock` y un control `ImageEx` que enlaza sus propiedades `Text` y `Source` a las propiedades respectivas de nuestro modelo `Post`, y un sencillo `TextBlock` dentro de un `ScrollView` que se usa para mostrar el contenido (ejemplo) de la publicación seleccionada.

Aplicación de ejemplo

Aplicación de ejemplo disponible [aquí](#).

¡Todo listo!

Ahora hemos creado todos nuestros modelos de vista, los servicios necesarios y la interfaz de usuario para nuestros widgets: nuestro sencillo explorador de Reddit está completado. Esto estaba pensado para ser un ejemplo de cómo compilar una aplicación siguiendo el patrón MVVM y usando las API del kit de herramientas de MVVM.

Como se indicó anteriormente, esto es solo una referencia y puede modificar esta estructura para adaptarse a sus necesidades o elegir solo un subconjunto de componentes de la biblioteca. Independientemente del enfoque que tome, el kit de herramientas de MVVM debe proporcionar una base sólida para empezar a trabajar desde cero al iniciar una nueva aplicación, ya que le permite centrarse en la lógica de negocios en lugar de tener que preocuparse por realizar manualmente todas las labores necesarias para permitir el soporte adecuado para el patrón MVVM.

Last updated on 07/11/2024

Migración desde MvvmLight

Artículo • 14/03/2024

En este artículo se describen algunas de las principales diferencias entre el [kit de herramientas MvvmLight](#) y el kit de herramientas de MVVM a la hora de facilitar la migración.

Aunque este artículo se centra específicamente en las migraciones de MvvmLight al kit de herramientas de MVVM, tenga en cuenta que hay mejoras adicionales que se han realizado en el kit de herramientas de MVVM, por lo que es muy recomendable echar un vistazo a la documentación de las nuevas API individuales.

API de la plataforma: [ObservableObject](#), [ObservableRecipient](#), [RelayCommand](#), [RelayCommand<T>](#), [AsyncRelayCommand](#), [AsyncRelayCommand<T>](#), [IMessenger](#), [WeakReferenceMessenger](#), [StrongReferenceMessenger](#), [IRecipient<TMessage>](#), [MessageHandler<TRecipient, TMessage>](#), [IMessengerExtensions](#)

Instalación del kit de herramientas de MVVM

Para aprovechar el kit de herramientas de MVVM, primero deberá instalar el paquete NuGet más reciente en la aplicación .NET existente.

Instalación mediante la CLI de .NET

```
dotnet add package CommunityToolkit.Mvvm --version 8.1.0
```

Instalación mediante PackageReference

XML

```
<PackageReference Include="CommunityToolkit.Mvvm" Version="8.1.0" />
```

Migración de ObservableObject

Los pasos siguientes se centran en migrar los componentes existentes que aprovechan las ventajas de `ObservableObject` del kit de herramientas de MvvmLight. El kit de herramientas de MVVM proporciona un tipo de [ObservableObject](#) similar.

El primer cambio aquí será intercambiar el uso de directivas en los componentes.

C#

```
// MvvmLight
using GalaSoft.MvvmLight;

// MVVM Toolkit
using CommunityToolkit.Mvvm.ComponentModel;
```

A continuación se muestra una lista de migraciones que deberán realizarse si se usan en la solución actual.

Métodos ObservableObject

Set<T>(Expression, ref T, T)

Set(Expression, ref T, T) no tiene un reemplazo de firma de método similar.

Sin embargo, SetProperty(ref T, T, string) proporciona la misma funcionalidad con ventajas de rendimiento adicionales.

C#

```
// MvvmLight
Set(() => MyProperty, ref this.myProperty, value);

// MVVM Toolkit
SetProperty(ref this.myProperty, value);
```

Tenga en cuenta que el parámetro `string` no es necesario si se llama al método desde el establecedor de la propiedad, ya que se deduce del nombre del miembro llamador, tal y como se puede ver aquí. Si desea invocar `SetProperty` para una propiedad diferente de la cuyo método se invoca, puede hacerlo mediante el operador `nameof`, lo que puede resultar útil para que el código sea menos propenso a errores al no tener nombres codificados de forma rígida. Por ejemplo:

C#

```
SetProperty(ref this.someProperty, value, nameof(SomeProperty));
```

Set<T>(string, ref T, T)

Set<T>(string, ref T, T) no tiene un reemplazo de firma de método similar.

Sin embargo, `SetProperty<T>(ref T, T, string)` proporciona la misma funcionalidad con parámetros ordenados de nuevo.

C#

```
// MvvmLight
Set(nameof(MyProperty), ref this.myProperty, value);

// MVVM Toolkit
SetProperty(ref this.myProperty, value);
```

`Set<T>(ref T, T, string)`

`Set<T>(ref T, T, string)` tiene un reemplazo directo cuyo nombre se ha cambiado, `SetProperty<T>(ref T, T, string)`.

C#

```
// MvvmLight
Set(ref this.myProperty, value, nameof(MyProperty));

// MVVM Toolkit
SetProperty(ref this.myProperty, value);
```

`RaisePropertyChanged(string)`

`RaisePropertyChanged(string)` tiene un reemplazo directo cuyo nombre se ha cambiado, `OnPropertyChanged(string)`.

C#

```
// MvvmLight
RaisePropertyChanged(nameof(MyProperty));

// MVVM Toolkit
OnPropertyChanged();
```

Al igual que con `SetProperty`, el método `OnPropertyChanged` deduce automáticamente el nombre de la propiedad actual. Si desea usar este método para generar manualmente el evento `PropertyChanged` para otra propiedad, también puede especificar manualmente de nuevo el nombre de esa propiedad mediante el operador `nameof`. Por ejemplo:

C#

```
OnPropertyChanged(nameof(SomeProperty));
```

RaisePropertyChanged<T>(Expression)

RaisePropertyChanged<T>(Expression) no tiene un reemplazo directo.

Para mejorar el rendimiento, se recomienda que reemplace `RaisePropertyChanged<T>(Expression)` por `OnPropertyChanged(string)` del kit de herramientas mediante la palabra clave `nameof` en su lugar (o sin parámetros, si la propiedad de destino es la misma que la que llama al método, por lo que el nombre se puede deducir automáticamente tal y como se mencionó antes).

C#

```
// MvvmLight
RaisePropertyChanged(() => MyProperty);

// MVVM Toolkit
OnPropertyChanged(nameof(MyProperty));
```

VerifyPropertyName(string)

No hay ningún reemplazo directo para el método `VerifyPropertyName(string)` y cualquier código que lo use debe modificarse o quitarse.

El motivo de la omisión del kit de herramientas de MVVM es que el uso de la palabra clave `nameof` para una propiedad verifica que esta exista. Cuando se creó `MvvmLight`, la palabra clave `nameof` no estaba disponible y este método se usó para asegurar que la propiedad existía en el objeto.

C#

```
// MvvmLight
VerifyPropertyName(nameof(MyProperty));

// MVVM Toolkit
// No direct replacement, remove
```

Propiedades de ObservableObject

PropertyChangedHandler

`PropertyChangedHandler` no tiene un reemplazo directo.

Para generar un evento con cambio de propiedad a través del controlador de eventos `PropertyChanged`, debe llamar al método `OnPropertyChanged` en su lugar.

C#

```
// MvvmLight
PropertyChangedEventHandler handler = PropertyChangedHandler;

// MVVM Toolkit
OnPropertyChanged();
```

Migrando ViewModelBase

Los pasos siguientes se centran en migrar los componentes existentes que aprovechan las ventajas de `ViewModelBase` del kit de herramientas de MvvmLight.

El kit de herramientas de MVVM proporciona un tipo [ObservableRecipient](#) que brinda una funcionalidad similar.

A continuación se muestra una lista de migraciones que deberán realizarse si se usan en la solución actual.

Métodos ViewModelBase

`Set<T>(string, ref T, T, bool)`

`Set<T>(string, ref T, T, bool)` no tiene un reemplazo de firma de método similar.

Sin embargo, `SetProperty<T>(ref T, T, bool, string)` proporciona la misma funcionalidad con parámetros ordenados de nuevo.

C#

```
// MvvmLight
Set(nameof(MyProperty), ref this.myProperty, value, true);

// MVVM Toolkit
SetProperty(ref this.myProperty, value, true);
```

Tenga en cuenta que el valor y los parámetros booleanos de difusión no son opcionales en la implementación del kit de herramientas de MVVM y deben proporcionarse para usar este

método. El motivo de este cambio es que al omitir el parámetro de difusión al llamar a este método, se llamará de forma predeterminada al método `SetProperty` de `ObservableObject`.

Tenga también en cuenta que el parámetro `string` no es necesario si se llama al método desde el establecedor de la propiedad, ya que se deduce del nombre del miembro llamador, como en los métodos en la base de la clase `ObservableObject`.

`Set<T>(ref T, T, bool, string)`

`Set<T>(ref T, T, bool, string)` tiene un reemplazo directo cuyo nombre se ha cambiado, `SetProperty<T>(ref T, T, bool, string)`.

C#

```
// MvvmLight
Set(ref this.myProperty, value, true, nameof(MyProperty));

// MVVM Toolkit
SetProperty(ref this.myProperty, value, true);
```

`Set<T>(Expression, ref T, T, bool)`

`Set<T>(Expression, ref T, T, bool)` no tiene un reemplazo directo.

Se recomienda para mejorar el rendimiento que reemplace por el `SetProperty<T>(ref T, T, bool, string)` del kit de herramientas de MVVM mediante la palabra clave `nameof` en su lugar.

C#

```
// MvvmLight
Set<MyObject>(() => MyProperty, ref this.myProperty, value, true);

// MVVM Toolkit
SetProperty(ref this.myProperty, value, true);
```

`Broadcast<T>(T, T, string)`

`Broadcast<T>(T, T, string)` tiene un reemplazo directo que no requiere un cambio de nombre.

C#


```
// MvvmLight
Broadcast<MyObject>(oldValue, newValue, nameof(MyProperty));

// MVVM Toolkit
Broadcast(oldValue, newValue, nameof(MyProperty));
```

Tenga en cuenta que el mensaje enviado a través de la propiedad `Messenger` al llamar al método `Broadcast` tiene un reemplazo directo para `PropertyChangedMessage` dentro de la biblioteca del kit de herramientas MVVM.

RaisePropertyChanged<T>(string, T, T, bool)

No hay ningún reemplazo directo para el método `RaisePropertyChanged<T>(string, T, T, bool)`.

La alternativa más sencilla es llamar a `OnPropertyChanged` y, posteriormente, llamar a `Broadcast` para obtener esta funcionalidad.

C#

```
// MvvmLight
RaisePropertyChanged<MyObject>(nameof(MyProperty), oldValue, newValue, true);

// MVVM Toolkit
OnPropertyChanged();
Broadcast(oldValue, newValue, nameof(MyProperty));
```

RaisePropertyChanged<T>(Expression, T, T, bool)

No hay ningún reemplazo directo para el método `RaisePropertyChanged<T>(Expression, T, T, bool)`.

La alternativa más sencilla es llamar a `OnPropertyChanged` y, posteriormente, llamar a `Broadcast` para obtener esta funcionalidad.

C#

```
// MvvmLight
RaisePropertyChanged<MyObject>(() => MyProperty, oldValue, newValue, true);

// MVVM Toolkit
OnPropertyChanged(nameof(MyProperty));
Broadcast(oldValue, newValue, nameof(MyProperty));
```

ICleanup.Cleanup()

No hay ningún reemplazo directo para la interfaz `ICleanup`.

Sin embargo, `ObservableRecipient` proporciona un método `OnDeactivated` que se debe usar para proporcionar la misma funcionalidad que `Cleanup`.

`OnDeactivated` en el kit de herramientas de MVVM también se anulará el registro de todos los eventos de messenger registrados cuando se les llame.

C#

```
// MvvmLight
Cleanup();

// MVVM Toolkit
OnDeactivated();
```

Tenga en cuenta que se puede llamar a los métodos `OnActivated` y `OnDeactivated` desde la solución existente como con `Cleanup`.

Sin embargo, `ObservableRecipient` expone una propiedad `IsActive` que también controla la llamada a estos métodos cuando se establece.

Propiedades de ViewModelBase

MessengerInstance

`MessengerInstance` tiene un reemplazo directo cuyo nombre se ha cambiado, `Messenger`.

C#

```
// MvvmLight
IMessenger messenger = MessengerInstance;

// MVVM Toolkit
IMessenger messenger = Messenger;
```

ⓘ Nota

El valor predeterminado de la propiedad `Messenger` será la instancia `WeakReferenceMessenger.Default`, que es la implementación estándar del mensajero de referencia parcial en el kit de herramientas de MVVM. Esto se puede personalizar

insertando una instancia de `IMessenger` diferente en el constructor de `ObservableRecipient`.

IsInDesignMode

No hay ningún reemplazo directo para la propiedad `IsInDesignMode` y cualquier código que lo use debe modificarse o quitarse.

El motivo de la omisión del kit de herramientas de MVVM es que la propiedad `IsInDesignMode` expone implementaciones específicas de la plataforma. El kit de herramientas de MVVM se ha diseñado para ser independiente a plataformas.

C#

```
// MvvmLight
var isInDesignMode = IsInDesignMode;

// MVVM Toolkit
// No direct replacement, remove
```

Propiedades estáticas de ViewModelBase

IsInDesignModeStatic

No hay ningún reemplazo directo para la propiedad `IsInDesignModeStatic` y cualquier código que lo use debe modificarse o quitarse.

El motivo de la omisión del kit de herramientas de MVVM es que la propiedad `IsInDesignMode` expone implementaciones específicas de la plataforma. El kit de herramientas de MVVM se ha diseñado para ser independiente a plataformas.

C#

```
// MvvmLight
var isInDesignMode = ViewModelBase.IsInDesignModeStatic;

// MVVM Toolkit
// No direct replacement, remove
```

Migración de RelayCommand

Los pasos siguientes se centran en migrar los componentes existentes que aprovechan las ventajas de `RelayCommand` del kit de herramientas de MvvmLight.

El kit de herramientas de MVVM proporciona un tipo de `RelayCommand` que proporciona una funcionalidad similar que aprovecha la interfaz del sistema de `ICommand` .

A continuación se muestra una lista de migraciones que deberán realizarse si se usan en la solución actual. Cuando no aparece un método o una propiedad, hay un reemplazo directo con el mismo nombre en el kit de herramientas de MVVM y no se requiere ningún cambio.

El primer cambio aquí será intercambiar el uso de directivas en los componentes.

C#

```
// MvvmLight
using GalaSoft.MvvmLight.Command;
using GalaSoft.MvvmLight.CommandWpf;

// MVVM Toolkit
using CommunityToolkit.Mvvm.Input;
```

⚠ Nota

MvvmLight usa referencias parciales para establecer el vínculo entre el comando y la acción a la que se llama desde la clase asociada. La implementación del kit de herramientas de MVVM no requiere esto y si este parámetro opcional se ha establecido como `true` en cualquiera de los constructores, se quitará.

Uso de RelayCommand con acciones asincrónicas

Si actualmente usa la implementación de MvvmLight `RelayCommand` con acciones asincrónicas, el kit de herramientas de MVVM expone una implementación mejorada para estos escenarios.

Simplemente puede reemplazar el `RelayCommand` existente por el `AsyncRelayCommand` que se ha creado con fines asincrónicos.

C#

```
// MvvmLight
var command = new RelayCommand(() => OnCommandAsync());
var command = new RelayCommand(async () => await OnCommandAsync());

// MVVM Toolkit
var asyncCommand = new AsyncRelayCommand(OnCommandAsync);
```

Métodos RelayCommand

RaiseCanExecuteChanged()

La funcionalidad de `RaiseCanExecuteChanged()` se puede lograr con el método `NotifyCanExecuteChanged()` del kit de herramientas MVVM.

C#

```
// MvvmLight
var command = new RelayCommand(OnCommand);
command.RaiseCanExecuteChanged();

// MVVM Toolkit
var command = new RelayCommand(OnCommand);
command.NotifyCanExecuteChanged();
```

Migración de RelayCommand<T>

Los pasos siguientes se centran en migrar los componentes existentes que aprovechan las ventajas de `RelayCommand<T>` del kit de herramientas de MvvmLight.

El kit de herramientas de MVVM proporciona un tipo de `RelayCommand<T>` que proporciona una funcionalidad similar que aprovecha la interfaz del sistema de `ICommand`.

A continuación se muestra una lista de migraciones que deberán realizarse si se usan en la solución actual. Cuando no aparece un método o una propiedad, hay un reemplazo directo con el mismo nombre en el kit de herramientas de MVVM y no se requiere ningún cambio.

El primer cambio aquí será intercambiar el uso de directivas en los componentes.

C#

```
// MvvmLight
using GalaSoft.MvvmLight.Command;
using GalaSoft.MvvmLight.CommandWpf;

// MVVM Toolkit
using CommunityToolkit.Mvvm.Input;
```

Uso de RelayCommand con comandos asincrónicos

Si actualmente usa la implementación de MvvmLight `RelayCommand<T>` con acciones asíncronas, el kit de herramientas de MVVM expone una implementación mejorada para estos escenarios.

Simplemente puede reemplazar el `RelayCommand<T>` existente por el `AsyncRelayCommand<T>` que se ha creado con fines asíncronos.

C#

```
// MvvmLight
var command = new RelayCommand<string>(async () => await OnCommandAsync());

// MVVM Toolkit
var asyncCommand = new AsyncRelayCommand<string>(OnCommandAsync);
```

`RelayCommand<T>` Métodos

`RaiseCanExecuteChanged()`

La funcionalidad de `RaiseCanExecuteChanged()` se puede lograr con el método `NotifyCanExecuteChanged()` del kit de herramientas MVVM.

C#

```
// MvvmLight
var command = new RelayCommand<string>(OnCommand);
command.RaiseCanExecuteChanged();

// MVVM Toolkit
var command = new RelayCommand<string>(OnCommand);
command.NotifyCanExecuteChanged();
```

Migración de `SimpleIoc`

La implementación `IoC` en el kit de herramientas de MVVM no incluye ninguna lógica integrada para controlar la inserción de dependencias por su cuenta, por lo que puede usar cualquier biblioteca de terceros para recuperar una instancia de `IServiceProvider` que pueda pasar al método `Ioc.ConfigureServices`. En los ejemplos siguientes, se usará el tipo `ServiceCollection` de la biblioteca de `Microsoft.Extensions.DependencyInjection`.

Este es el mayor cambio entre MvvmLight y el kit de herramientas de MVVM.

Esta implementación resultará familiar si ha implementado la inserción de dependencias con aplicaciones de ASP.NET Core.

Registro de dependencias

Con MvvmLight, es posible que haya registrado las dependencias similares a estos escenarios mediante `SimpleIoc`.

C#

```
public void RegisterServices()
{
    SimpleIoc.Default.Register<INavigationService, NavigationService>();

    SimpleIoc.Default.Register<IDialogService>(() => new DialogService());
}
```

Con el kit de herramientas de MVVM, lograría lo mismo que se indica a continuación.

C#

```
public void RegisterServices()
{
    Ioc.Default.ConfigureServices(
        new ServiceCollection()
            .AddSingleton<INavigationService, NavigationService>()
            .AddSingleton<IDialogService>(new DialogService())
            .BuildServiceProvider());
}
```

Resolución de dependencias

Una vez inicializado, los servicios se pueden recuperar de la clase `Ioc` igual que con `SimpleIoc`:

C#

```
IDialogService dialogService = SimpleIoc.Default.GetInstance<IDialogService>();
```

Al migrar al kit de herramientas de MVVM, logrará lo mismo con:

C#

```
IDialogService dialogService = Ioc.Default.GetService<IDialogService>();
```

Eliminar dependencias

Con `SimpleIoc`, anularía el registro de las dependencias con la siguiente llamada de método.

C#

```
SimpleIoc.Default.Unregister<INavigationService>();
```

No hay ningún reemplazo directo para quitar dependencias con la implementación del kit de herramientas de MVVM `Ioc`.

Constructor preferido

Al registrar las dependencias con `SimpleIoc` de `MvvmLight`, puede optar por recibir clases para proporcionar un atributo `PreferredConstructor` para aquellos con varios constructores.

Este atributo debe quitarse dónde se usa y tendrá que usar los atributos de la biblioteca de inserción de dependencias de terceros en uso, si se admiten.

Migración de `Messenger`

Los pasos siguientes se centran en migrar los componentes existentes que aprovechan las ventajas de `Messenger` del kit de herramientas de `MvvmLight`.

El kit de herramientas de MVVM proporciona dos implementaciones de messenger (`WeakReferenceMessenger` y `StrongReferenceMessenger`, consulte los documentos [aquí](#)) que proporcionan una funcionalidad similar, con algunas diferencias clave que se detallan a continuación.

A continuación se muestra una lista de migraciones que deberán realizarse si se usan en la solución actual.

El primer cambio aquí será intercambiar el uso de directivas en los componentes.

C#

```
// MvvmLight
using GalaSoft.MvvmLight.Messaging;

// MVVM Toolkit
using CommunityToolkit.Mvvm.Messaging;
```


Métodos de Messenger

Register<TMessage>(object, Action<TMessage>)

La funcionalidad de `Register<TMessage>(object, Action<TMessage>)` se puede lograr con el método `IMessenger` del kit de herramientas MVVM `Register<TRecipient, TMessage>(object, MessageHandler<TRecipient, TMessage>)`.

C#

```
// MvvmLight
Messenger.Default.Register<MyMessage>(this, this.OnMyMessageReceived);

// MVVM Toolkit
Messenger.Register<MyViewModel, MyMessage>(this, static (r, m) =>
    r.OnMyMessageReceived(m));
```

La razón de esta firma es que permite a messenger usar referencias parciales para realizar un seguimiento correcto de los destinatarios y evitar la creación de cierres para capturar el propio destinatario. Es decir, el destinatario de entrada se pasa como entrada a la expresión lambda, por lo que no es necesario capturarlo mediante la propia expresión lambda. Esto también da como resultado código más eficaz, ya que el mismo controlador se puede reutilizar varias veces sin asignaciones. Tenga en cuenta que esta es solo una de las formas admitidas para registrar controladores, y también es posible usar la interfaz `IRecipient<TMessage>` en su lugar (detalles [en los documentos de messenger](#)), lo que hace que el registro sea automático y menos detallado.

⚠ Nota

El modificador `static` para expresiones lambda requiere C# 9 y es opcional. Es útil usarlo aquí para asegurarse de que no captura accidentalmente el destinatario o algún otro miembro provocando la asignación de un cierre, pero no es obligatorio. Si no puede usar C# 9, solo tiene que quitar `static` aquí y tener cuidado de asegurarse de que el código no captura nada.

Además, este ejemplo y los siguientes simplemente usarán la propiedad `Messenger` de `ObservableRecipient`. Si desea acceder de forma estática a una instancia de Messenger desde cualquier otro lugar del código, también se aplican los mismos ejemplos, con la única diferencia de que `Messenger` debe reemplazarse por p. ej. `WeakReferenceMessenger.Default` en su lugar.

Register<TMessage>(object, bool, Action<TMessage>)

No hay ningún reemplazo directo para este mecanismo de registro que le permita admitir también la recepción de mensajes para los tipos de mensajes derivados. Este cambio es intencionado, ya que la implementación de `Messenger` tiene como objetivo no usar la reflexión para lograr las ventajas de rendimiento.

Como alternativa, hay algunas opciones que se pueden llevar a cabo para lograr esta funcionalidad.

- Crear una implementación `IMessenger` personalizada.
- Registre los tipos de mensaje adicionales mediante un controlador compartido que compruebe el tipo e invoque el método correcto.

C#

```
// MvvmLight
Messenger.Default.Register<MyMessage>(this, true, this.OnMyMessageReceived);

// MVVM Toolkit
Messenger.Register<MyViewModel, MyMessage>(this, static (r, m) =>
    r.OnMyMessageReceived(m));
Messenger.Register<MyViewModel, MyOtherMessage>(this, static (r, m) =>
    r.OnMyMessageReceived(m));
```

Register<TMessage>(object, object, Action<TMessage>)

La funcionalidad de `Register<TMessage>(object, object, Action<TMessage>)` se puede lograr con el método `Register<TRecipient, TMessage, TToken>(object, TToken, MessageHandler<TRecipient, TMessage>)` del kit de herramientas MVVM.

C#

```
// MvvmLight
Messenger.Default.Register<MyMessage>(this, nameof(MyViewModel),
    this.OnMyMessageReceived);

// MVVM Toolkit
Messenger.Register<MyViewModel, MyMessage, string>(this, nameof(MyViewModel),
    static (r, m) => r.OnMyMessageReceived(m));
```

Register<TMessage>(object, object, bool, Action<TMessage>)

No hay ningún reemplazo directo para este mecanismo de registro que le permita admitir también la recepción de mensajes para los tipos de mensajes derivados. Este cambio es intencionado, ya que la implementación de `Messenger` tiene como objetivo no usar la reflexión para lograr las ventajas de rendimiento.

Como alternativa, hay algunas opciones que se pueden llevar a cabo para lograr esta funcionalidad.

- Crear una implementación `IMessenger` personalizada.
- Registre los tipos de mensaje adicionales mediante un controlador compartido que compruebe el tipo e invoque el método correcto.

C#

```
// MvvmLight
Messenger.Default.Register<MyMessage>(this, nameof(MyViewModel), true,
this.OnMyMessageReceived);

// MVVM Toolkit
Messenger.Register<MyViewModel, MyMessage, string>(this, nameof(MyViewModel),
static (r, m) => r.OnMyMessageReceived(m));
Messenger.Register<MyViewModel, MyOtherMessage, string>(this, nameof(MyViewModel),
static (r, m) => r.OnMyMessageReceived(m));
```

`Send<TMessage>(TMessage)`

La funcionalidad de `Send<TMessage>(TMessage)` se puede lograr con el método de extensión `Send<TMessage>(TMessage)` del kit de herramientas MVVM `IMessenger`.

C#

```
// MvvmLight
Messenger.Default.Send<MyMessage>(new MyMessage());
Messenger.Default.Send(new MyMessage());

// MVVM Toolkit
Messenger.Send(new MyMessage());
```

En el escenario anterior en el que el mensaje que se envía tiene un constructor sin parámetros, el kit de herramientas de MVVM tiene una extensión simplificada para enviar un mensaje en este formato.

C#

```
// MVVM Toolkit
```

```
Messenger.Send<MyMessage>();
```

Send<TMessage>(TMessage, object)

La funcionalidad de `Send<TMessage>(TMessage, object)` se puede lograr con el método `Send<TMessage, TToken>(TMessage, TToken)` del kit de herramientas MVVM.

C#

```
// MvvmLight
Messenger.Default.Send<MyMessage>(new MyMessage(), nameof(MyViewModel));
Messenger.Default.Send(new MyMessage(), nameof(MyViewModel));

// MVVM Toolkit
Messenger.Send(new MyMessage(), nameof(MyViewModel));
```

Unregister(object)

La funcionalidad de `Unregister(object)` se puede lograr con el método `UnregisterAll(object)` del kit de herramientas MVVM.

C#

```
// MvvmLight
Messenger.Default.Unregister(this);

// MVVM Toolkit
Messenger.UnregisterAll(this);
```

Unregister<TMessage>(object)

La funcionalidad de `Unregister<TMessage>(object)` se puede lograr con el método de extensión `Unregister<TMessage>(object)` del kit de herramientas MVVM `IMessenger`.

C#

```
// MvvmLight
Messenger.Default.Unregister<MyMessage>(this);

// MVVM Toolkit
Messenger.Unregister<MyMessage>(this);
```

Unregister<TMessage>(object, Action<TMessage>)

No hay ningún reemplazo directo para el método `Unregister<TMessage>(object, Action<TMessage>)` en el kit de herramientas MVVM.

El motivo de la omisión es que un destinatario del mensaje solo puede tener un único controlador registrado para cualquier tipo de mensaje determinado.

Se recomienda lograr esta funcionalidad con el método de extensión `Unregister<TMessage>(object)` del kit de herramientas MVVM `IMessenger`.

C#

```
// MvvmLight
Messenger.Default.Unregister<MyMessage>(this, OnMyMessageReceived);

// MVVM Toolkit
Messenger.Unregister<MyMessage>(this);
```

Unregister<TMessage>(object, object)

La funcionalidad de `Unregister<TMessage>(object, object)` se puede lograr con el método `Unregister<TMessage, TToken>(object, TToken)` del kit de herramientas MVVM.

C#

```
// MvvmLight
Messenger.Default.Unregister<MyMessage>(this, nameof(MyViewModel));

// MVVM Toolkit
Messenger.Unregister<MyMessage, string>(this, nameof(MyViewModel));
```

Unregister<TMessage>(object, object, Action<TMessage>)

No hay ningún reemplazo directo para el método `Unregister<TMessage>(object, object, Action<TMessage>)` en el kit de herramientas MVVM.

El motivo de la omisión es que un destinatario del mensaje solo puede tener un único controlador registrado para cualquier tipo de mensaje determinado.

Se recomienda lograr esta funcionalidad con el método del kit de herramientas MVVM `Unregister<TMessage, TToken>(object, TToken)`.

C#

```
// MvvmLight
Messenger.Default.Unregister<MyMessage>(this, nameof(MyViewModel),
OnMyMessageReceived);

// MVVM Toolkit
Messenger.Unregister<MyMessage, string>(this, nameof(MyViewModel));
```

Cleanup()

El método `Cleanup` tiene un reemplazo directo con el mismo nombre en el kit de herramientas MVVM. Tenga en cuenta que este método solo es útil cuando se usa un messenger que use referencias parciales, mientras que el tipo de `StrongReferenceMessenger` no hará nada cuando se llame a este método, ya que el estado interno ya se recortará automáticamente a medida que se use el messenger.

C#

```
// MvvmLight
Messenger.Default.Cleanup();

// MVVM Toolkit
Messenger.Cleanup();
```

RequestCleanup()

No hay ningún reemplazo directo para el método `RequestCleanup` en el kit de herramientas MVVM. En el contexto de `MvvmLight`, se usa `RequestCleanup` para iniciar una solicitud para quitar registros que ya no estén activos, ya que la implementación aprovecha las referencias parciales.

Las llamadas al método `RequestCleanup` se pueden quitar o reemplazar por `Cleanup`.

C#

```
// MvvmLight
Messenger.Default.RequestCleanup();

// MVVM Toolkit
// No direct replacement, remove
```

ResetAll()

La funcionalidad de `ResetAll()` se puede lograr con el método `Reset()` del kit de herramientas MVVM.

A diferencia de la implementación de `MvvmLight` que anula la instancia, el kit de herramientas MVVM borra los mapas registrados.

C#

```
// MvvmLight
Messenger.Default.ResetAll();

// MVVM Toolkit
Messenger.Reset();
```

Métodos estáticos de messenger

`OverrideDefault(IMessenger)`

No hay ningún reemplazo directo para el método `OverrideDefault(IMessenger)` en el kit de herramientas MVVM.

Para usar una implementación personalizada de `IMessenger`, registre la implementación personalizada en los registros de servicio para la inserción de dependencias o cree manualmente una instancia estática y pase esto cuando sea necesario.

C#

```
// MvvmLight
Messenger.OverrideDefault(new Messenger());

// MVVM Toolkit
// No direct replacement
```

`Reset()`

No hay ningún reemplazo directo para el método estático `Reset` en el kit de herramientas MVVM.

La misma funcionalidad se puede lograr llamando al método `Reset` de la instancia `Default` estática de uno de los tipos messenger.

C#

```
// MvvmLight
Messenger.Reset();

// MVVM Toolkit
WeakReferenceMessenger.Default.Reset();
```

Propiedades estáticas de messenger

Default

`Default` tiene un reemplazo directo, `Default`, que no requiere ningún cambio en la implementación existente.

```
C#

// MvvmLight
IMessenger messenger = Messenger.Default;

// MVVM Toolkit
IMessenger messenger = WeakReferenceMessenger.Default;
```

Migración de tipos de mensaje

Los tipos de mensaje proporcionados en el kit de herramientas de MvvmLight están diseñados como base para que usted como desarrollador trabaje con ellos si es necesario.

Aunque el kit de herramientas de MVVM proporciona algunas alternativas, no hay ningún reemplazo directo para estos tipos de mensaje. Se recomienda examinar nuestros [tipos de mensaje disponibles](#) [↗](#).

Como alternativa, si la solución aprovecha los tipos de mensaje MvvmLight, estos se pueden migrar fácilmente a su propio código base.

Migración de componentes específicos de la plataforma

En la implementación actual del kit de herramientas de MVVM, no hay reemplazos para componentes específicos de la plataforma que existan en el kit de herramientas de MvvmLight.

Los siguientes componentes y sus métodos auxiliares o de extensión asociados no tienen un reemplazo y tendrán que tenerse en cuenta al migrar al kit de herramientas de MVVM.

Específico de Android/iOS/Windows

- `DialogService`
- `DispatcherHelper`
- `NavigationService`

Específico de Android/iOS

- `ActivityBase`
- `Binding`
- `BindingMode`
- `PropertyChangedEventManager`
- `UpdateTriggerMode`

Específico de Android

- `CachingViewHolder`
- `ObservableAdapter`
- `ObservableRecyclerAdapter`

Específico de iOS

- `ObservableCollectionViewSource`
- `ObservableTableViewController`
- `ObservableTableViewSource`

Asistentes

- `Empty`
- `WeakAction`
- `WeakFunc`

Migración desde MVVM Básico

Artículo • 07/11/2024

En este artículo se explica cómo migrar aplicaciones compiladas con la opción [MVVM](#) Básico en [Windows Template Studio](#) para usar la biblioteca del kit de herramientas de MVVM en su lugar. Se aplica a las aplicaciones de UWP y WPF creadas con Windows Template Studio.

API de la plataforma: [ObservableObject](#), [RelayCommand](#)

Este artículo se centra exclusivamente en la migración y no trata cómo usar las funcionalidades adicionales que proporciona la biblioteca.

Instalación del kit de herramientas de MVVM

Para usar el kit de herramientas de MVVM, debe instalar el paquete NuGet en la aplicación existente.

Instalación mediante la CLI de .NET

```
dotnet add package CommunityToolkit.Mvvm --version 8.1.0
```

Instalación mediante PackageReference

XML

```
<PackageReference Include="CommunityToolkit.Mvvm" Version="8.1.0" />
```

Actualización de un proyecto

Hay cuatro pasos para migrar el código generado por Windows Template Studio.

1. Elimine los archivos antiguos.
2. Reemplace el uso de `Observable`.
3. Agregue nuevas referencias de espacio de nombres.
4. Actualice los métodos con nombres diferentes.

1. Eliminación de los archivos antiguos

MVVM Básico consta de dos archivos.

C#

```
\Helpers\Observable.cs  
\Helpers\RelayCommand.cs
```

Elimine ambos archivos.

Si intenta compilar el proyecto en este momento, verá muchos errores. Estos errores pueden ser útiles para identificar los archivos que requieren cambios.

2. Reemplazo del uso de `Observable`

La clase `Observable` se usó como clase base para ViewModels. El kit de herramientas de MVVM contiene una clase similar con funcionalidad adicional denominada [ObservableObject](#).

Cambie todas las clases heredadas previamente de `Observable` para que se hereden de `ObservableObject`.

Por ejemplo

C#

```
public class MainViewModel : Observable
```

El código anterior se convertirá en:

C#

```
public class MainViewModel : ObservableObject
```

3. Adición de nuevas referencias de espacio de nombres

Agregue una referencia al espacio de nombres `CommunityToolkit.Mvvm.ComponentModel` en todos los archivos donde hay una referencia a `ObservableObject`.

Puede agregar manualmente la directiva adecuada o mover el cursor al objeto `ObservableObject` y presionar `Ctrl+.` para acceder al menú de acción rápida para agregarla.

C#

```
using CommunityToolkit.Mvvm.ComponentModel;
```

Agregue una referencia al espacio de nombres `CommunityToolkit.Mvvm.Input` en todos los archivos donde hay una referencia a [RelayCommand](#).

Puede agregar manualmente la directiva adecuada o mover el cursor al objeto `RelayCommand` y presionar `Ctrl+.` para acceder al menú de acción rápida para agregarla.

C#

```
using CommunityToolkit.Mvvm.Input;
```

4. Actualización de los métodos con nombres diferentes

Hay dos métodos que se deben actualizar para permitir nombres diferentes para la misma funcionalidad.

Todas las llamadas a `Observable.Set` deben reemplazarse por llamadas a `ObservableObject.SetProperty`.

Por lo tanto,

C#

```
set { Set(ref _elementTheme, value); }
```

El código anterior se convertirá en:

C#

```
set { SetProperty(ref _elementTheme, value); }
```

Todas las llamadas a `RelayCommand.OnCanExecuteChanged` deben reemplazarse por llamadas a `RelayCommand.NotifyCanExecuteChanged`.

Por lo tanto,

C#

```
(UndoCommand as RelayCommand)?.OnCanExecuteChanged();
```

El código anterior se convertirá en:

C#

```
(UndoCommand as RelayCommand)?.NotifyCanExecuteChanged();
```

La aplicación ahora debería funcionar con la misma funcionalidad que antes.